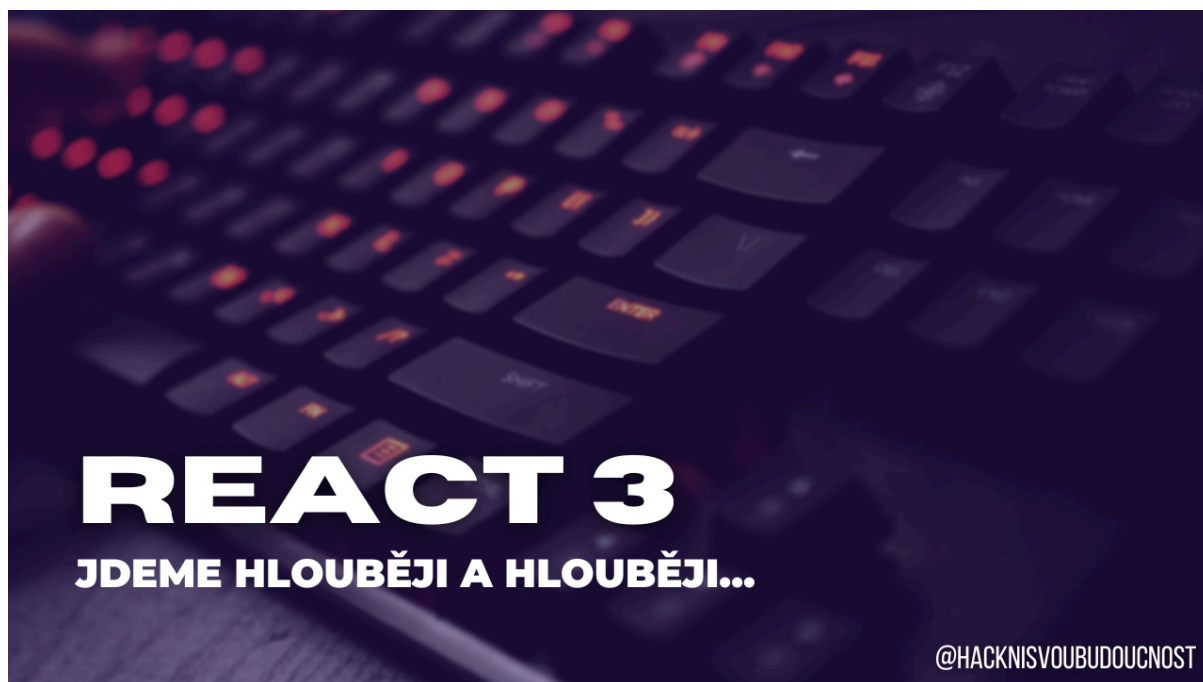


# React 3

k videím na YT Hackni svou budoucnost

David Šetek



Odkaz na učebnici

<https://bit.ly/hackni-react-3>

Seznam videí na kanále YouTube

David Šetek - Hackni svou budoucnost

[https://www.youtube.com/watch?v=Nz5LGoay5R4&list=PLQ8x\\_VWW6AkvURLFWrOZy1e8cZuyx\\_xN8](https://www.youtube.com/watch?v=Nz5LGoay5R4&list=PLQ8x_VWW6AkvURLFWrOZy1e8cZuyx_xN8)

# 1. React 3 - React router a vícestránkový web - tvoříme jednotlivé stránky

Video: <https://youtu.be/Nz5LGoay5R4>

Nainstalujeme React router. Tento příkaz spouštíme ve složce, kde spouštíme příkaz npm start.

```
npm install react-router-dom@6
```

## App.js

Naimportujeme BrowserRouter, Routes a Route.

BrowserRouter nás spojí s prohlížečem

Routes je obalující element pro Route

Route má dvě props = path a element

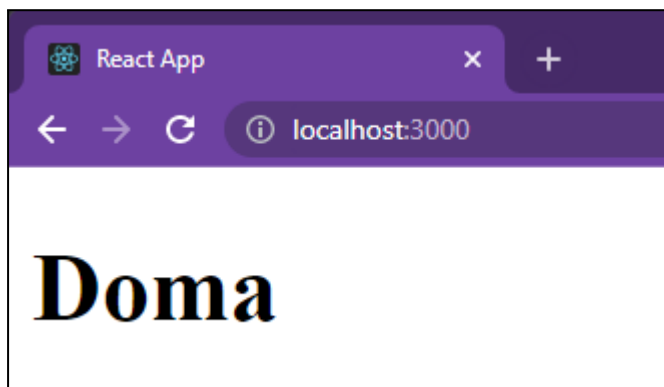
Do path dáváme název adresy a do element umístíme, co se má na stránce zobrazit.

```
import { BrowserRouter, Routes, Route } from "react-router-dom"

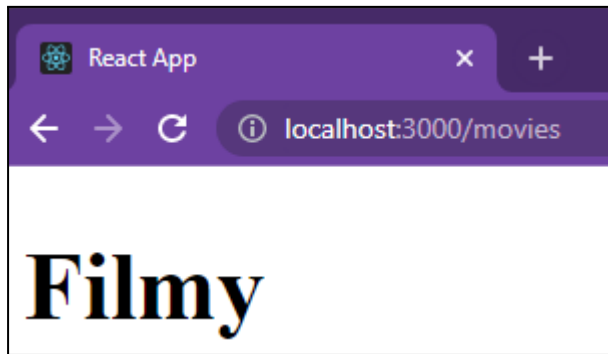
const App = () => {
  return <BrowserRouter>
    <Routes>
      <Route path="/" element={<h1>Doma</h1>} />
      <Route path="/movies" element={<h1>Filmy</h1>} />
      <Route path="/serials" element={<h1>Seriály</h1>} />
      <Route path="/new" element={<h1>Nové</h1>} />
    </Routes>
  </BrowserRouter>
}

export default App
```

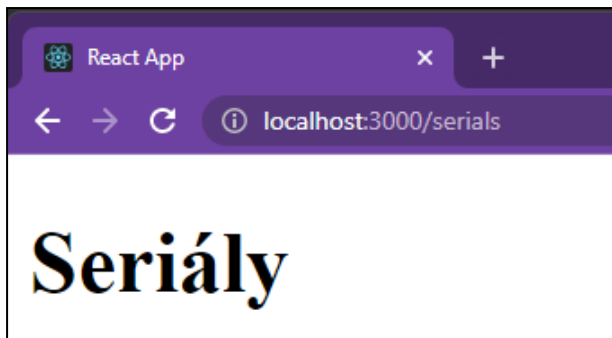
Zobrazení na stránce - adresa s lomítkem



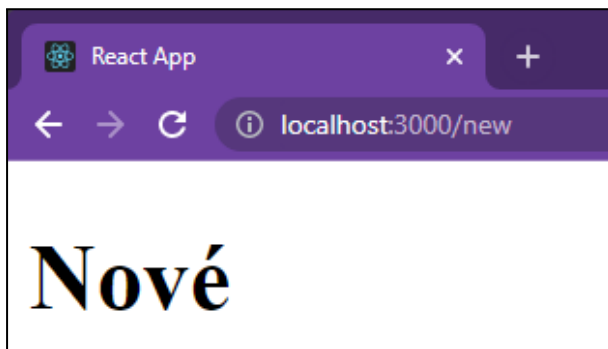
Adresa /movies



Adresa /serials



Adresa /new



## 2. React 3 - React router a vícestránkový web - více obsahu do stránky

Video: <https://youtu.be/5l26FO1M7aE>

### App.js

Pokud chceme na stránce větší množství obsahu, tak ho můžeme napsat pomocí tagů rovnou do element. Ale tím se soubor App.js stane nepřehledný. V následujícím videu si ukážeme, že můžeme zvolit elegantnější řešení, kde každá stránka bude mít svůj vlastní soubor.

```
import { BrowserRouter, Routes, Route } from "react-router-dom"

const App = () => {
  return <BrowserRouter>
    <Routes>
      <Route path="/" element={<h1>Doma</h1>} />
      <Route path="/movies" element={<h1>Filmy</h1>} />
      <Route path="/serials" element={<h1>Seriály</h1>} />
      <Route path="/new" element={<div>
        <h1>Sledujte nově</h1>
        <h2>Seriály</h2>
        <p>Lorem ipsum dolor sit amet, consectetur adipisicing elit. At
quod autem repellendus quis deleniti odio neque soluta saepe molestiae,
mollitia aut veniam quas nemo ullam! Odio neque perferendis explicabo
doloribus.</p>

        <h2>Filmy</h2>
        <p>Lorem ipsum dolor sit amet consectetur adipisicing elit.
Nisi quos perspiciatis amet aut corrupti, culpa, beatae velit
distinctio sequi nostrum minus quae possimus repellendus obcaecati
similique unde quaerat architecto deserunt!</p>
      </div>} />
    </Routes>
  </BrowserRouter>
}

export default App
```

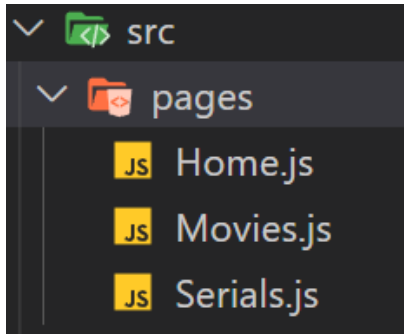
Výsledek na stránce /new



### 3. React 3 - React router a vícestránkový web - obsah stránek do souborů

Video: <https://youtu.be/rvKPs64IBm4>

Ve složce src vytvoříme složku pages a naplníme ji soubory. Každý soubor představuje jednu stránku. My budeme uvažovat tři stránky - úvodní, movies a serials.



Do všech souborů umístíme obyčejné komponenty.

Home.js

```
const Home = () => {  
  return (  
    <div>Domovská stránka</div>  
  )  
}  
  
export default Home
```

Movies.js

```
const Movies = () => {  
  return (  
    <div>Filmy</div>  
  )  
}  
  
export default Movies
```

Serials.js

```
const Serials = () => {  
  return (  
    <div>Seriály</div>  
  )  
}  
  
export default Serials
```

## App.js

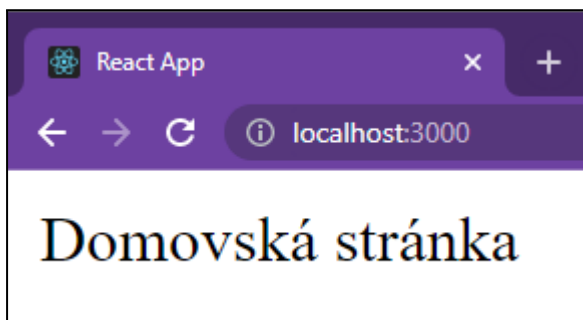
Do App.js naimportujeme všechny komponenty a umístíme je do element u každé stránky.

```
import { BrowserRouter, Routes, Route } from "react-router-dom"
import Home from "../pages/Home"
import Movies from "../pages/Movies"
import Serials from "../pages/Serials"

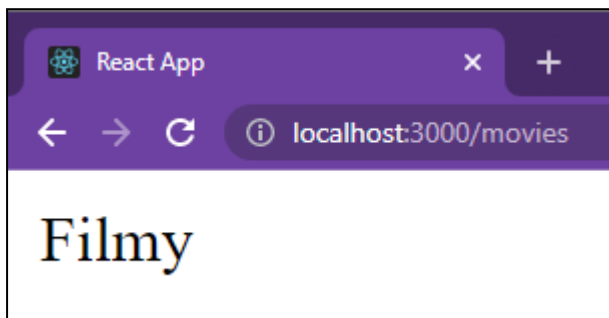
const App = () => {
  return <BrowserRouter>
    <Routes>
      <Route path="/" element={ <Home /> } />
      <Route path="/movies" element={ <Movies /> } />
      <Route path="/serials" element={ <Serials /> } />
    </Routes>
  </BrowserRouter>
}

export default App
```

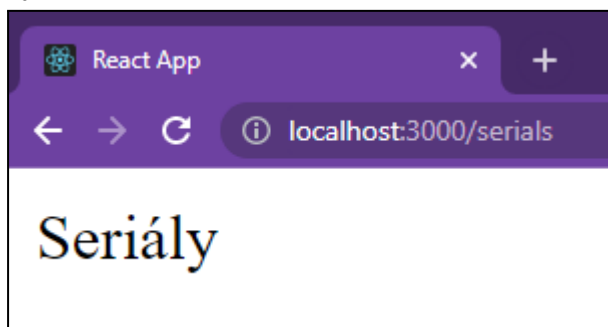
Výsledek na stránce /



Výsledek na stránce /movies



Výsledek na stránce /serials



## 4. React 3 - React router a vícestránkový web - link a propojení stránek

Video: <https://youtu.be/PMuT3R1mxzA>

Do všech souborů naimportujeme Link a ten použijeme k propojení stránek. Link má props s názvem "to", kam vždy píšeme, kam má odkaz směřovat.

Home.js

```
import { Link } from "react-router-dom"

const Home = () => {
  return <section>
    <ul>
      <li>Domovská stránka</li>
      <li><Link to="/movies">Filmy</Link></li>
      <li><Link to="/serials">Seriály</Link></li>
    </ul>
  </section>
}

export default Home
```

Movies.js

```
import { Link } from "react-router-dom"

const Movies = () => {
  return <section>
    <ul>
      <li><Link to="/">Domovská stránka</Link></li>
      <li>Filmy</li>
      <li><Link to="/serials">Seriály</Link></li>
    </ul>
  </section>
}

export default Movies
```

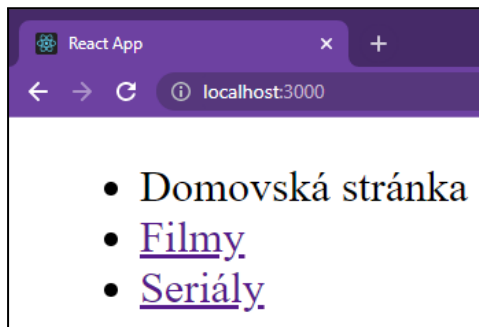
## Serials.js

```
import { Link } from "react-router-dom"

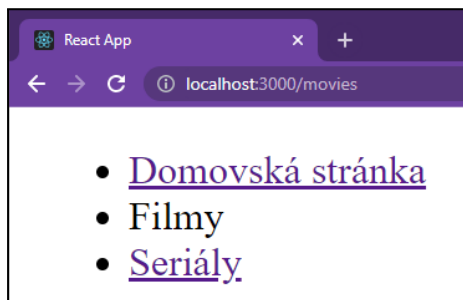
const Serials = () => {
  return <section>
    <ul>
      <li><Link to="/">Domovská stránka</Link></li>
      <li><Link to="/movies">Filmy</Link></li>
      <li>Seriály</li>
    </ul>
  </section>
}

export default Serials
```

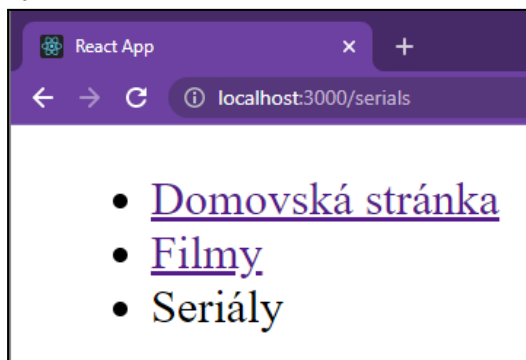
## Výsledek na stránce /



## Výsledek na stránce /movies



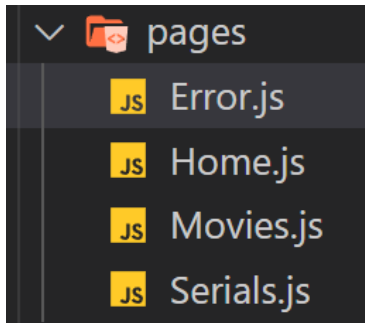
Výsledek na stránce /serials



## 5. React 3 - React router a vícestránkový web - neexistující stránka 404

Video: <https://youtu.be/fES6qOoR-Pc>

Ve složce pages vytvoříme soubor Error.js



### Error.js

Je to obyčejný komponenta a je na nás, čím ji naplníme. My jsme zde dali h2 a p a k tomu přidali ještě odkaz na úvodní stranu.

```
import { Link } from "react-router-dom"

const Error = () => {
  return <div>
    <h2>404</h2>
    <p>Stránka nenalezena</p>
    <p><Link to="/">Úvodní strana</Link></p>
  </div>
}

export default Error
```

### App.js

Naimportujeme komponentu Error a použijeme ji jako další Route. Hvězdička říká, že pokud někdo zadá jinou stránku než jsou ty výše (/, "movies", "serials"), tak se zobrazí obsah v Error.js

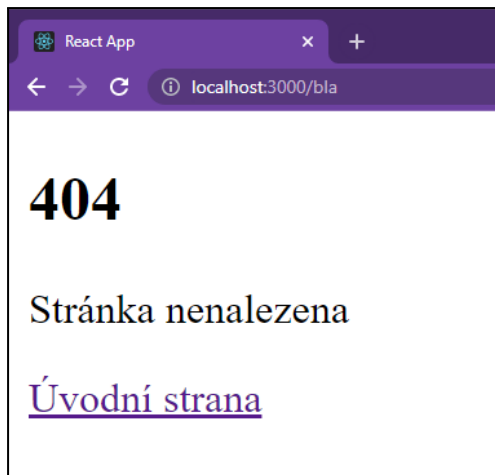
```
import { BrowserRouter, Routes, Route } from "react-router-dom"
import Home from "../pages/Home"
import Movies from "../pages/Movies"
import Serials from "../pages/Serials"
import Error from "../pages/Error"

const App = () => {
  return <BrowserRouter>
    <Routes>
      <Route path="/" element={ <Home /> } />
      <Route path="/movies" element={ <Movies /> } />
    </Routes>
  </BrowserRouter>
}
```

```
    <Route path="/serials" element={ <Serials /> } />
    <Route path="*" element={ <Error /> } />
  </Routes>
</BrowserRouter>
}

export default App
```

Výsledek na jakékoliv neexistující stránce. Zde na stránce /bla



## 6. React 3 - React router a vícestránkový web - hlavička a patička stránky

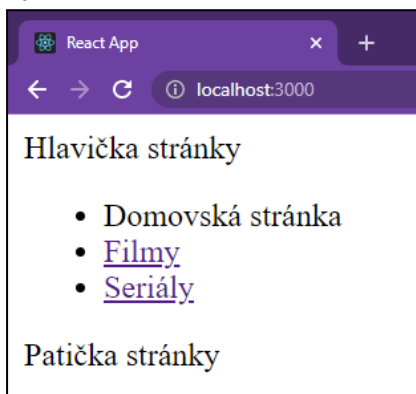
Video: <https://youtu.be/SbDJJ-XvC1M>

### App.js

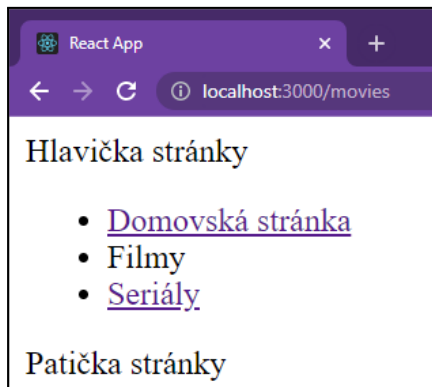
Nejjednodušší způsob, jak vytvořit hlavičku a patičku, kterou chcete mít na všech stránkách, je ten, že přidáme header a footer rovnou do App.js.

```
const App = () => {  
  return <BrowserRouter>  
    <header>Hlavička stránky</header>  
    <Routes>  
      <Route path="/" element={ <Home /> } />  
      <Route path="/movies" element={ <Movies /> } />  
      <Route path="/serials" element={ <Serials /> } />  
      <Route path="*" element={ <Error /> } />  
    </Routes>  
    <footer>Patička stránky</footer>  
  </BrowserRouter>  
}
```

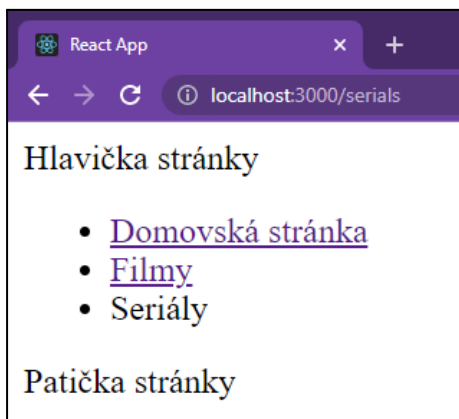
Výsledek na stránce /



Výsledek na stránce /movies



Výsledek na stránce /serials



## 7. React 3 - React router a vícestránkový web - obalující Route (nested routes) a outlet

Video: <https://youtu.be/DNN5cViqe34>

### App.js

Z route s komponentou Home uděláme na chvíli obalující element.

```
const App = () => {
  return <BrowserRouter>
    <header>Hlavička stránky</header>
    <Routes>
      <Route path="/" element={ <Home /> }>
        <Route path="/movies" element={ <Movies /> } />
        <Route path="/serials" element={ <Serials /> } />
        <Route path="*" element={ <Error /> } />
      </Route>
    </Routes>
    <footer>Patička stránky</footer>
  </BrowserRouter>
}

export default App
```

### Home.js

Protože Home obaluje všechny ostatní Route, tak pokud někdo chce vnitřní Route, tak se vezme obsah jeho komponenty a pod označením Outlet se to vloží do Home. Pokud někdo chce tedy stránku /movies, tak se vezme obsah komponenty Movies a vloží se do Home jako Outlet a vypíše se do stránky. Proto na stránce /movies najdete toto li s textem.

```
import { Link, Outlet } from "react-router-dom"

const Home = () => {
  return <section>
    <ul>
      <li>Domovská stránkaaaaaaaaaaaaaaaaa</li>
      <Outlet />
    </ul>
  </section>
}

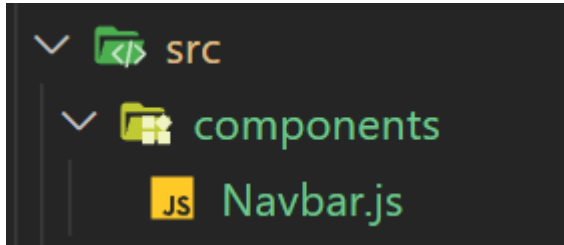
export default Home
```

Movies.js a Serials.js zůstaly beze změny

## 8. React 3 - React router a vícestránkový web - navigace v samostatném souboru

Video: [https://youtu.be/BBSldzm\\_cNA](https://youtu.be/BBSldzm_cNA)

Ve složce src vytvoříme složku components a v ní soubor Navbar.js



### Navbar.js

Obsahem bude navigace. Proto použijeme obalující tag <nav>, který obaluje právě navigaci. Uvnitř navigace budou odkazy (linky) na naše tři stránky.

```
import { Link } from "react-router-dom"

const Navbar = () => {
  return <nav>
    <Link to="/">Domů</Link><br />
    <Link to="/movies">Filmy</Link><br />
    <Link to="/serials">Seriály</Link>
  </nav>
}

export default Navbar
```

### Home.js

Navbar naimportujeme jako komponentu do Home.js a předěláme obsah return.

```
import { Link, Outlet } from "react-router-dom"
import Navbar from "../components/Navbar"

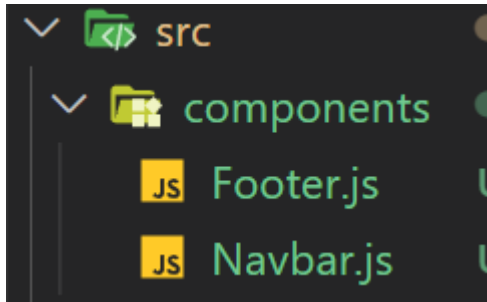
const Home = () => {
  return <>
    <Navbar />
    <section>
      <Outlet />
    </section>
  </>
}

export default Home
```

## 9. React 3 - React router a vícestránkový web - patička v samostatném souboru

Video: <https://youtu.be/eeMyaN7ILKw>

Ve složce src vytvoříme soubor Footer.js



### Footer.js

Vytvoříme komponentu patičky.

```
const Footer = () => {  
  return <footer>  
    <p>Patička</p>  
  </footer>  
}  
  
export default Footer
```

### Home.js

Komponentu Footer naimportujeme do Home.js a použijeme ji v return.

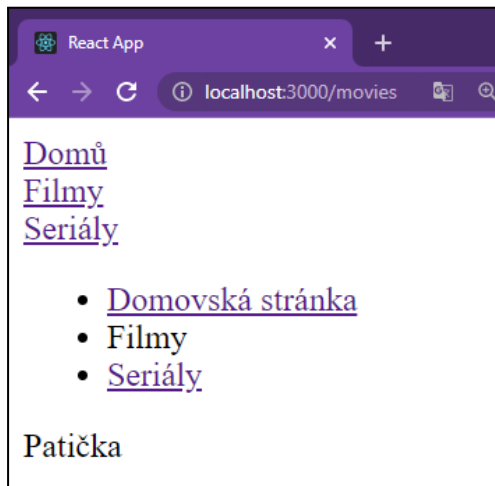
```
import { Link, Outlet } from "react-router-dom"  
import Navbar from "../components/Navbar"  
import Footer from "../components/Footer"  
  
const Home = () => {  
  return <>  
    <Navbar />  
    <section>  
      <Outlet />  
    </section>  
    <Footer />  
  </>  
}  
  
export default Home
```

## App.js

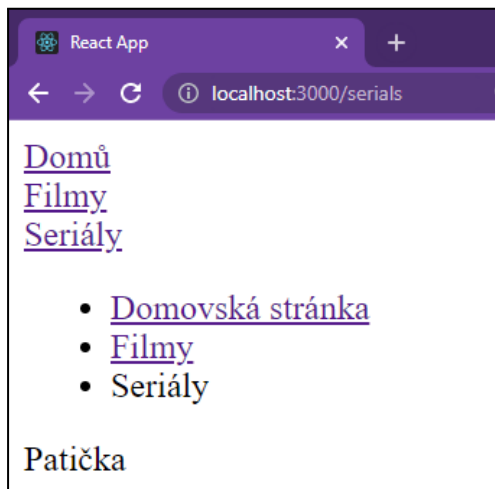
Smažeme tagy <header> a <footer>

```
const App = () => {  
  return <BrowserRouter>  
    <Routes>  
      <Route path="/" element={ <Home /> }>  
        <Route path="/movies" element={ <Movies /> } />  
        <Route path="/serials" element={ <Serials /> } />  
        <Route path="*" element={ <Error /> } />  
      </Route>  
    </Routes>  
  </BrowserRouter>  
}
```

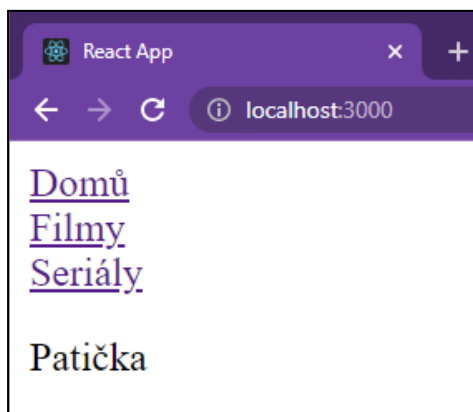
## Stránka /movies



## Stránka /serials



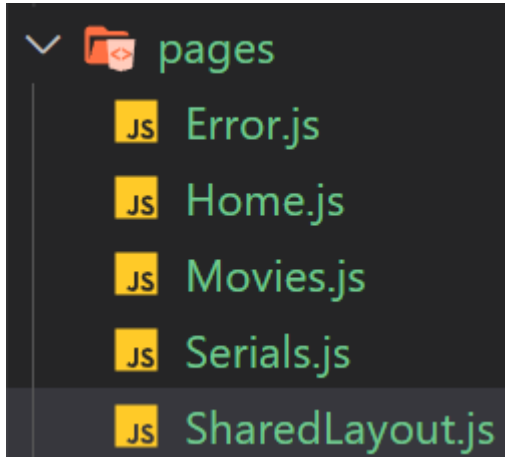
Stránka /



## 10. React 3 - React router a vícestránkový web - sdílený layout všech stránek

Video: <https://youtu.be/oimgzhWh65U>

Ve složce pages vytvoříme soubor SharedLayout.js



### **SharedLayout.js**

Vytvoříme novou komponentu SharedLayout. Jedná se o sdílený layout do všech našich stránek.

```
import { Outlet } from "react-router-dom"
import Navbar from "../components/Navbar"
import Footer from "../components/Footer"

const SharedLayout = () => {
  return <>
    <Navbar />
    <Outlet />
    <Footer />
  </>
}

export default SharedLayout
```

## App.js

Naimportujeme SharedLayout a použijeme v return. U komponenty Home použijeme slovo index. To značí, že se má použít path obalující elementu (tedy path="/")

```
import { BrowserRouter, Routes, Route } from "react-router-dom"
import Home from "../pages/Home"
import Movies from "../pages/Movies"
import Serials from "../pages/Serials"
import Error from "../pages/Error"
import SharedLayout from "../pages/SharedLayout"

const App = () => {
  return <BrowserRouter>
    <Routes>

      <Route path="/" element={ <SharedLayout /> }>
        <Route index element={ <Home /> } />
        <Route path="/movies" element={ <Movies /> } />
        <Route path="/serials" element={ <Serials /> } />
        <Route path="*" element={ <Error /> } />
      </Route>

    </Routes>
  </BrowserRouter>
}

export default App
```

### Home.js

```
import { Link, Outlet } from "react-router-dom"
import Navbar from "../components/Navbar"
import Footer from "../components/Footer"

const Home = () => {
  return <section>
    <h1>Úvodní strana</h1>
  </section>
}

export default Home
```

### Movies.js

```
import { Link } from "react-router-dom"

const Movies = () => {
  return <section>
    <h1>Filmy</h1>
  </section>
}

export default Movies
```

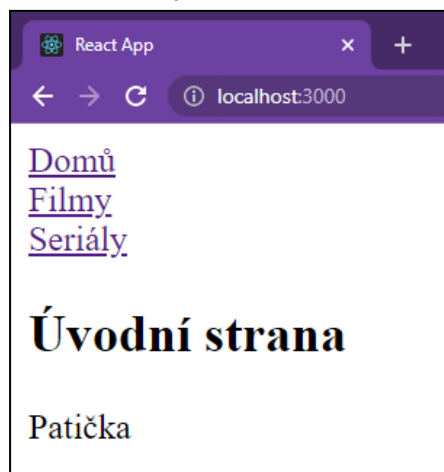
### Serials.js

```
import { Link } from "react-router-dom"

const Serials = () => {
  return <section>
    <h1>Seriály</h1>
  </section>
}

export default Serials
```

Takto např. vypadá úvodní strana /



## 11. React 3 - React router a vícestránkový web - úklid v souborech

Video: <https://youtu.be/lcoZDaRU4uQ>

### Navbar.js

Předěláme strukturu, aby měla blíže ke klasické struktuře header, obsah stránky, footer.

```
import { Link } from "react-router-dom"

const Navbar = () => {
  return <header>
    <nav>
      <Link to="/">Domů</Link><br />
      <Link to="/movies">Filmy</Link><br />
      <Link to="/serials">Seriály</Link>
    </nav>
  </header>
}

export default Navbar
```

### Home.js - co je růžovou barvou, tak to jsme smazali

```
import { Link, Outlet } from "react-router-dom"
import Navbar from "../components/Navbar"
import Footer from "../components/Footer"

const Home = () => {
  return <section>
    <h1>Úvodní strana</h1>
  </section>
}

export default Home
```

### Movies.js - co je růžovou barvou, tak to jsme smazali

```
import { Link } from "react-router-dom"

const Movies = () => {
  return <section>
    <h1>Filmy</h1>
  </section>
}

export default Movies
```

**Serials.js** - co je růžovou barvou, tak to jsme smazali

```
import { Link } from "react-router-dom"
```

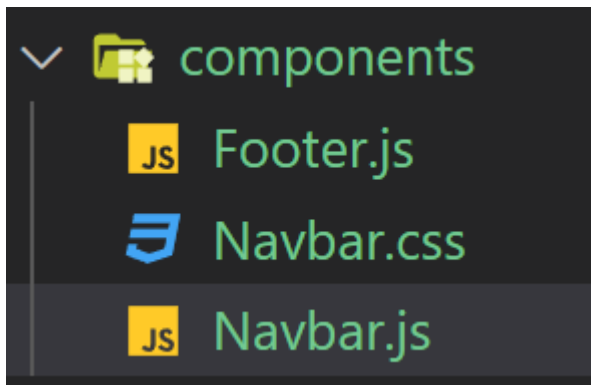
```
const Serials = () => {  
  return <section>  
    <h1>Seriály</h1>  
  </section>  
}
```

```
export default Serials
```

## 12. React 3 - React router a vícestránkový web - NavLink

Video: <https://youtu.be/P43XboVWS6k>

Ve složce components si založíme soubor Navbar.css



### Navbar.js

Napojíme Navbar.css do Navbar.js (zelený kód). Místo Link použijeme NavLink. Ten dělá to, že pokud je stránka aktivní (je na ní v prohlížeči kliknuto), tak přidám classu active (automaticky, my jsme ji nikde nenastavovali).

```
import { NavLink } from "react-router-dom"
import "./Navbar.css"

const Navbar = () => {
  return <header>
    <nav>
      <NavLink to="/">Domů</NavLink><br />
      <NavLink to="/movies">Filmy</NavLink><br />
      <NavLink to="/serials">Seriály</NavLink>
    </nav>
  </header>
}

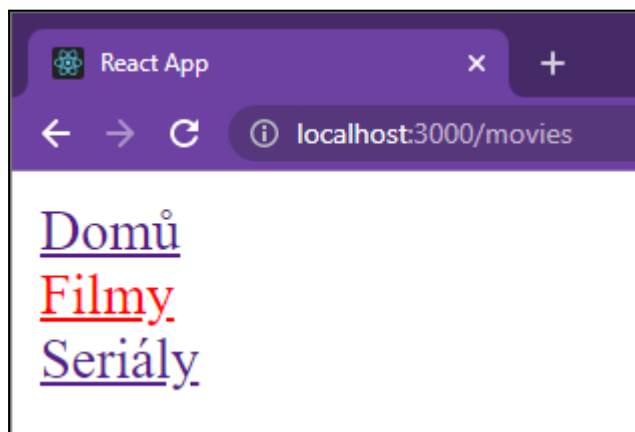
export default Navbar
```

### Navbar.css

Zde vezmeme classu active a obarvíme text červeně.

```
.active {
  color: red;
}
```

Pokud je kliknut např. na stránku /movies, tak dostává classu active a text se obarví červeně.



## 13. React 3 - React router a vícestránkový web - isActive a pokročilé stylování

Video: <https://youtu.be/0uCPgpHrNg8>

### Navbar.js

NavLink dělá i to, že posílá do proměnné isActive hodnoty true nebo false podle toho, jestli je odkaz v prohlížeči aktivní nebo ne (je na něj kliknuto nebo ne). Pokud je Domů kliknuto, tak se isActive naplní hodnotou true. Dále máme podmínku (zapsanou ternárním operátorem), že pokud je isActive true, tak přidej className activeLink. Pokud je false, tak přidej classu nonactiveLink.

```
import { NavLink } from "react-router-dom"
import "./Navbar.css"

const Navbar = () => {
  return <header>
    <nav>
      <NavLink to="/" className={({ isActive }) =>
        isActive ? "activeLink" : "nonactiveLink"
      } >Domů</NavLink><br />

      <NavLink to="/movies" className={({ isActive }) =>
        isActive ? "activeLink" : "nonactiveLink"
      }>Filmy</NavLink><br />

      <NavLink to="/serials" className={({ isActive }) =>
        isActive ? "activeLink" : "nonactiveLink"
      } >Seriály</NavLink>
    </nav>
  </header>
}

export default Navbar
```

### Navbar.css

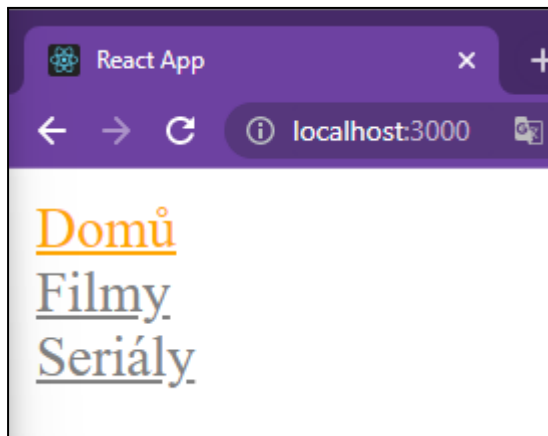
Zde mám stanovené, co mají classy activeLink a nonactiveLink provádět - zde jen obarvují text na oranžovou a šedou barvu.

```
.activeLink {
  color: orange;
}

.nonactiveLink {
  color: grey;
```

```
}
```

Výsledek v prohlížeči.



## 14. React 3 - React router a vícestránkový web - parametry v URL adrese

Video: <https://youtu.be/-cjdXNwgZAQ>

### OneMovie.js

Vytvoříme běžnou komponentu a dovnitř dáme h2 s libovolným textem. Zde je text "Jeden film".

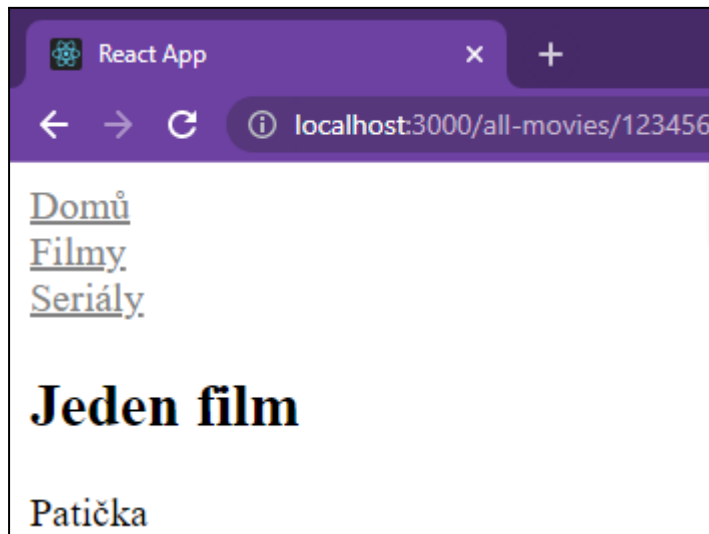
```
const OneMovie = () => {  
  return <section>  
    <h2>Jeden film</h2>  
  </section>  
}  
  
export default OneMovie
```

### App.js

Komponentu OneMovie naimportujeme do App.js. Vytvoříme další stránku, na které se OneMovie bude zobrazovat. Tato stránka bude mít ale v adrese ještě parametr s názvem movieId (název je libovolný).

```
import { BrowserRouter, Routes, Route } from "react-router-dom"  
import Home from "../pages/Home"  
import Movies from "../pages/Movies"  
import Serials from "../pages/Serials"  
import Error from "../pages/Error"  
import SharedLayout from "../pages/SharedLayout"  
import OneMovie from "../components/OneMovie"  
  
const App = () => {  
  return <BrowserRouter>  
    <Routes>  
  
      <Route path="/" element={ <SharedLayout /> }>  
        <Route index element={ <Home /> } />  
        <Route path="/movies" element={ <Movies /> } />  
        <Route path="/serials" element={ <Serials /> } />  
        <Route path="/all-movies/:movieId" element={ <OneMovie /> } />  
        <Route path="*" element={ <Error /> } />  
      </Route>  
  
    </Routes>  
  </BrowserRouter>  
}  
  
export default App
```

Výsledek na stránce, pokud zadáme adresu /all-movies/123456



Dodatek: pokud bychom chtěli do URL adresy více parametrů, tak to zapíšeme takto:

```
<Route path="/all-movies/:movieId/:category" element={ <OneMovie /> } />
```

## 15. React 3 - React router a vícestránkový web - načítáme parametry (useParams)

Video: <https://youtu.be/cbnOaaqWc4k>

### OneMovie.js

Pomocí useParams získáváme parametry z url adresy. Vráť parametry jako objekt, který má název a hodnotu. Název zde bude movieId (to máme v App.js) a hodnota bude to, co je zadané v url adrese. My si vypíšeme obsah movieId do return.

```
import { useParams } from "react-router-dom"

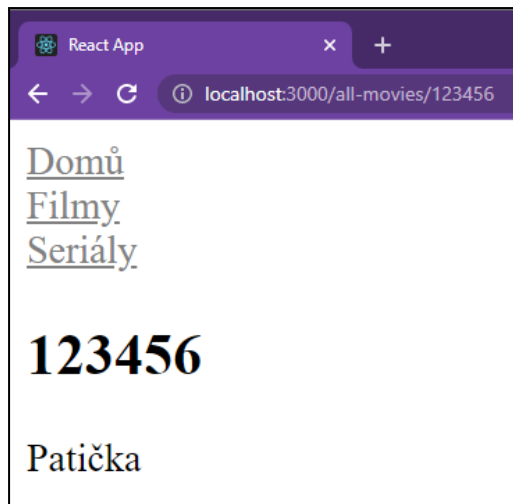
const OneMovie = () => {
  console.log(useParams())

  const {movieId} = useParams()

  return <section>
    <h2>{movieId}</h2>
  </section>
}

export default OneMovie
```

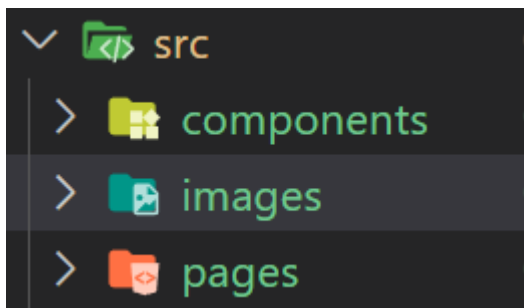
Výsledek na stránce, když se zadá adresa /all-movies/123456



## 16. React 3 - React router a vícestránkový web - generování stránek pro filmy

Video: <https://youtu.be/l7Bq73aTvDE>

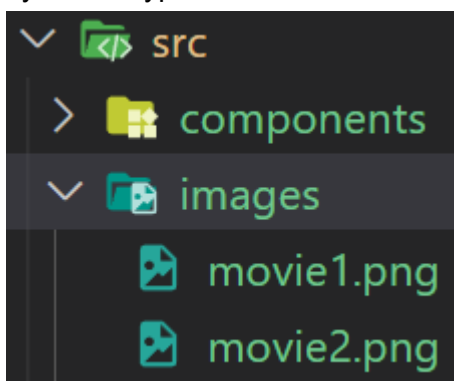
Ve složce src vytvoříme složku images.



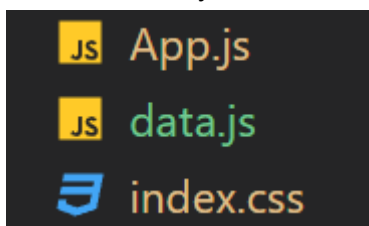
Naplníme ji obrázky, které najdete na tomto odkazu:

<https://drive.google.com/drive/folders/1td236jZt7gvk1WxD2YzB9Edot7AxMk2P?usp=sharing>

Výsledek vypadá takto.



Ve složce src vytvoříme soubor data.js



A naplníme ho těmito daty:

```
import image1 from "../images/movie1.png"
import image2 from "../images/movie2.png"
import image3 from "../images/movie3.png"
import image4 from "../images/movie4.png"
import image5 from "../images/movie5.png"
import image6 from "../images/movie6.png"
import image7 from "../images/movie7.png"
import image8 from "../images/movie8.png"
```

```
import image9 from "./images/movie9.png"
import image10 from "./images/movie10.png"

const allMovies = [
  {
    id: 1,
    image: image1,
    title: "Enola Holmesová 2",
    age: "+13",
    tags: "Vtipný * Napínavý * Rodina",
    description: "Lorem ipsum dolor sit, amet consectetur
adipisicing elit. Autem accusantium amet nemo minus quo architecto
quas",
  },
  {
    id: 2,
    image: image2,
    title: "Enola Holmesová",
    age: "+13",
    tags: "Vtipný * Napínavý * Rodina",
    description: "Lorem ipsum dolor sit, amet consectetur
adipisicing elit. Autem accusantium amet nemo minus quo architecto
quas",
  },
  {
    id: 3,
    image: image3,
    title: "Rocknrolla",
    age: "+16",
    tags: "Gangster * Napínavý * Násilí",
    description: "Lorem ipsum dolor sit, amet consectetur
adipisicing elit. Autem accusantium amet nemo minus quo architecto
quas",
  },
  {
    id: 4,
    image: image4,
    title: "Patriot",
    age: "+16",
    tags: "Gangster * Napínavý * Násilí",
    description: "Lorem ipsum dolor sit, amet consectetur
adipisicing elit. Autem accusantium amet nemo minus quo architecto
quas",
  },
]
```

```
},
{
  id: 5,
  image: image5,
  title: "Telefon pana Harrigana",
  age: "+16",
  tags: "Gangster * Napínavý * Násilí",
  description: "Lorem ipsum dolor sit, amet consectetur
adipisicing elit. Autem accusantium amet nemo minus quo architecto
quas",
},
{
  id: 6,
  image: image6,
  title: "Stážista",
  age: "+16",
  tags: "Rodinný * Napínavý * Romantický",
  description: "Lorem ipsum dolor sit, amet consectetur
adipisicing elit. Autem accusantium amet nemo minus quo architecto
quas",
},
{
  id: 7,
  image: image7,
  title: "Život",
  age: "+13",
  tags: "Vesmír * Napínavý * Nebezpečí",
  description: "Lorem ipsum dolor sit, amet consectetur
adipisicing elit. Autem accusantium amet nemo minus quo architecto
quas",
},
{
  id: 8,
  image: image8,
  title: "Kód za miliardu dolarů",
  age: "+13",
  tags: "IT * Napínavý",
  description: "Lorem ipsum dolor sit, amet consectetur
adipisicing elit. Autem accusantium amet nemo minus quo architecto
quas",
},
{
  id: 9,
```

```

      image: image9,
      title: "Pracháči",
      age: "+16",
      tags: "Dokument * Napínavý",
      description: "Lorem ipsum dolor sit, amet consectetur
adipisicing elit. Autem accusantium amet nemo minus quo architecto
quas",
    },
    {
      id: 10,
      image: image10,
      title: "Whoami",
      age: "+13",
      tags: "Gangster * Napínavý * Násilí",
      description: "Lorem ipsum dolor sit, amet consectetur
adipisicing elit. Autem accusantium amet nemo minus quo architecto
quas",
    }
  ]

export default allMovies

```

## Movies.js

Naimportujeme data, která necháme vypsat do stránky. U každého filmu bude odkaz s textem “Více informací”. Ten bude odkazovat na stránku /all-movies/IDzeSouboruData

```

import movies from "../data"
import { Link } from "react-router-dom"

function Movies() {
  return <section>
    <h1>Filmy</h1>
    <div>
      {movies.map( (oneMovie) => {
        return <article key={oneMovie.id}>
          <h2>{oneMovie.title}</h2>
          <img src={oneMovie.image} alt="" /><br />
          <Link to={`/all-movies/${oneMovie.id}`}>Více informací</Link>
        </article>
      } )}
    </div>
  </section>
}

```

```
}  
  
export default Movies
```

### OneMovie.js

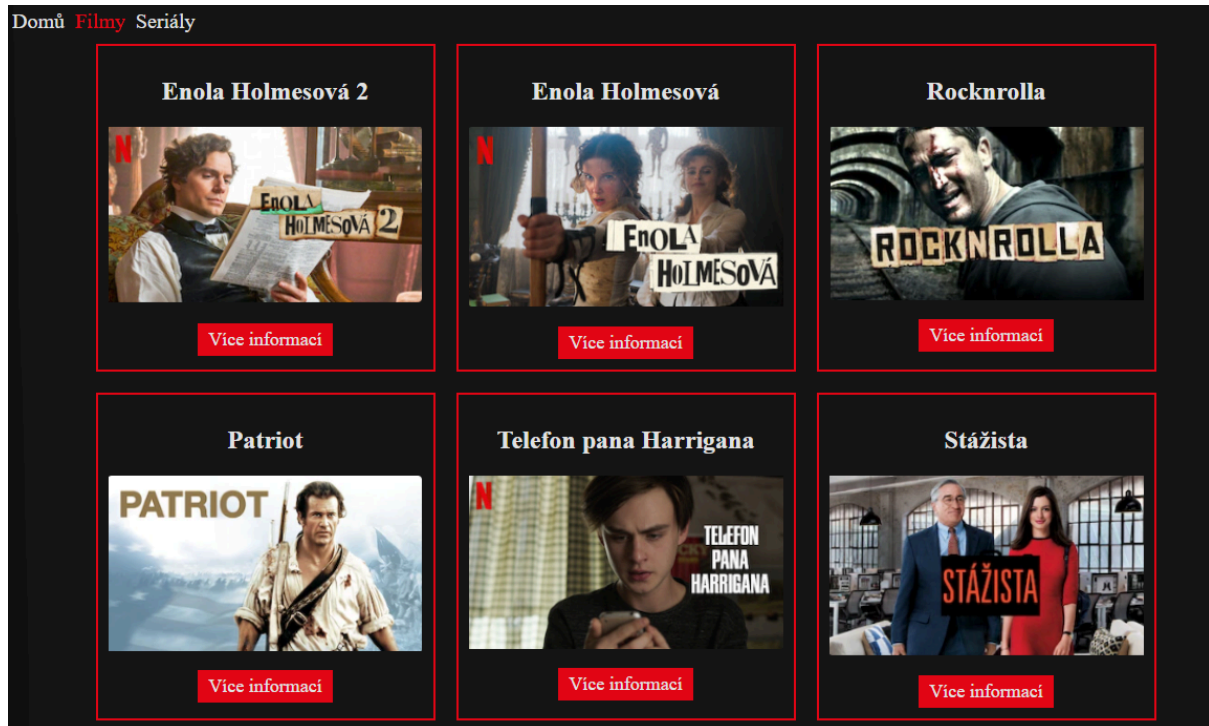
Vybereme pouze ten film, jehož ID se shoduje s ID, které získáme z URL adresy jako parametr. Tento film vypíšeme do stránky.

```
import { useParams } from "react-router-dom"  
import { Link } from "react-router-dom"  
import movies from "../data"  
  
const OneMovie = () => {  
  
  const {movieId} = useParams()  
  
  const oneSpecificMovie = movies.find( (oneMovie) => {  
    return oneMovie.id === parseInt(movieId)  
  })  
  
  const {image, title, description, tags} = oneSpecificMovie  
  
  return <section>  
    <h2>{title}</h2>  
    <img src={image} alt="" />  
    <p>{description}</p>  
    <p>{tags}</p>  
    <Link to="/movies">Zpět na všechny filmy</Link>  
  </section>  
}  
  
export default OneMovie
```

## 17. React 3 - React router a vícestránkový web - měníme texty a stylujeme

Video: [https://youtu.be/fyB8QM1\\_AJA](https://youtu.be/fyB8QM1_AJA)

Takto bude vypadat nejdůležitější stránka 😊



index.css

```
body {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
  background-color: #141414;
  color: #e5e5e5;
}
```

Movies.js

```
function Movies() {
  return <section>
    <div className="all-movies">
      {movies.map( (oneMovie) => {
        return <article key={oneMovie.id} className="one-movie">
          <h2>{oneMovie.title}</h2>
          <img src={oneMovie.image} alt="" /><br />
          <Link to={`/${all-movies}/${oneMovie.id}`}>Více informací</Link>
        </article>
      } )}
    </div>
  </section>
}
```

```

    </div>
  </section>
}

```

## Movies.css

```

.all-movies {
  display: flex;
  flex-direction: row;
  justify-content: center;
  flex-wrap: wrap;
}

.one-movie {
  width: 300px;
  border: 2px solid #e50914;
  display: flex;
  flex-direction: column;
  align-items: center;
  padding: 10px;
  margin: 10px;
  /* border-radius: 15px; */
}

.one-movie img {
  width: 100%;
}

.one-movie a {
  text-decoration: none;
  color: #e5e5e5;
  background-color: #e50914;
  padding: 5px 10px;
  font-size: 18px;
}

```

## Navbar.js

```

const Navbar = () => {
  return <header>
    <nav>
      <NavLink to="/" className={ ({isActive}) =>
        isActive ? "activeLink link" : "nonactiveLink link"
      } >Domů</NavLink>
    </nav>
  </header>
}

```

```

    <NavLink to="/movies" className={({ isActive }) =>
      isActive ? "activeLink link" : "nonactiveLink link"
    }>Filmy</NavLink>

    <NavLink to="/serials" className={({ isActive }) =>
      isActive ? "activeLink link" : "nonactiveLink link"
    } >Seriály</NavLink>
  </nav>
</header>
}

```

### Navbar.css

```

nav {
  margin-top: 10px;
  font-size: 20px;
}

.link {
  margin-left: 10px;
  text-decoration: none;
}

.activeLink {
  color: #e50914;;
}

.nonactiveLink {
  color: #e5e5e5;
}

```

### OneMovie.js

```

return <section className="one-movie-more">
  <h2>{title}</h2>
  <img src={image} alt="" />
  <p>{description}</p>
  <p>{tags}</p>
  <Link to="/movies">Zpět na všechny filmy</Link>
</section>

```

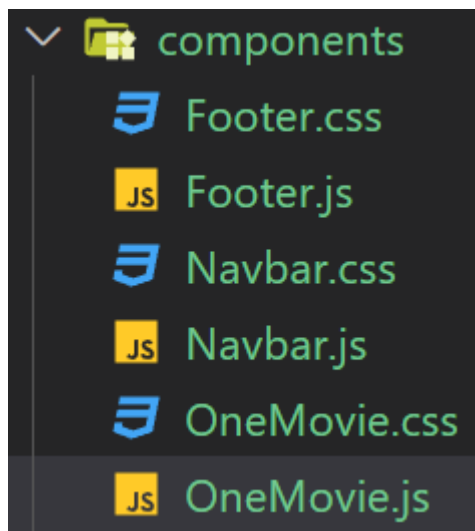
### OneMovie.css

```
.one-movie-more {  
  text-align: center;  
}  
  
.one-movie-more a {  
  text-decoration: none;  
  color: #e5e5e5;  
  background-color: #e50914;  
  padding: 5px 10px;  
  font-size: 18px;  
}
```

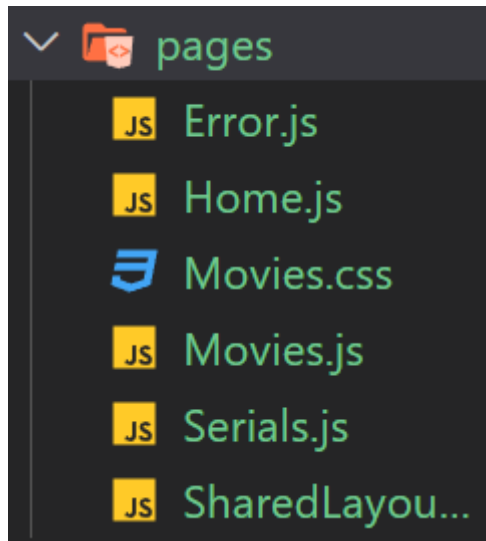
### Footer.css

```
footer p {  
  text-align: center;  
  font-size: 18px;  
}
```

### Složka components



Složka pages

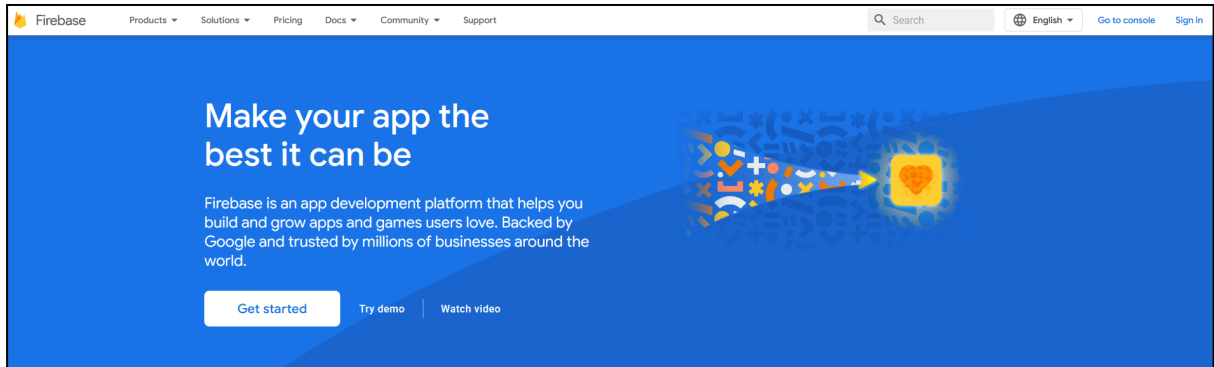


## 18. React 3 - Databáze firebase - úvod

Video: [https://youtu.be/PNA\\_5TPo51Y](https://youtu.be/PNA_5TPo51Y)

Na této stránce najdeme firebase:

<https://firebase.google.com/>

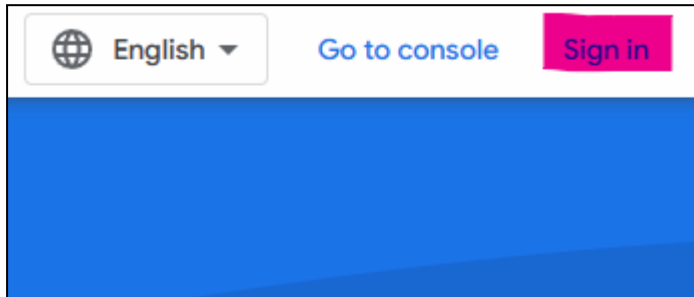


Založíme si běžný projekt ve Visual studio code. App.js a index.css budou prázdné.

## 19. React 3 - Databáze firebase - přihlášení a vytvoření projektu

Video: [https://youtu.be/Xo\\_1l1qjfyA](https://youtu.be/Xo_1l1qjfyA)

1. půjdeme na stránku: <https://firebase.google.com/>
2. v pravém horním rohu dáme Sign in, abychom si mohli vytvořit účet na firebase



3. po přihlášení se dostáváme na úvodní stranu, kde dáme Create project
4. zadáme název projektu (libovolný)

A screenshot of the 'Let's start with a name for your project' screen in the Firebase console. The title is 'Let's start with a name for your project' with a help icon. Below it, the 'Project name' field contains 'movies-project'. A preview of the project ID 'movies-project-8c464' is shown below the name field. At the bottom, there is a blue 'Continue' button.

## 5. vypneme Google Analytics a dáme Create project

# Google Analytics

## for your Firebase project

Google Analytics is a free and unlimited analytics solution that enables targeting, reporting, and more in Firebase Crashlytics, Cloud Messaging, In-App Messaging, Remote Config, A/B Testing, and Cloud Functions.

Google Analytics enables:

- ✕ A/B testing ?
- ✕ User segmentation & targeting across Firebase products ?
- ✕ Crash-free users ?
- ✕ Event-based Cloud Functions triggers ?
- ✕ Free unlimited reporting ?

 **Enable Google Analytics for this project**  
Recommended

[Previous](#)[Create project](#)

## 6. dostáváme se na úvodní stranu projektu

movies-project [Spark plan](#)

### Get started by adding Firebase to your app



Add an app to get started



Store and sync app data in milliseconds



**Authentication**  
Authenticate and manage users

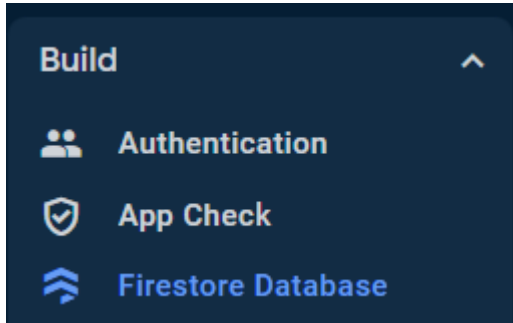


**Cloud Firestore**  
Realtime updates, powerful queries, and automatic scaling

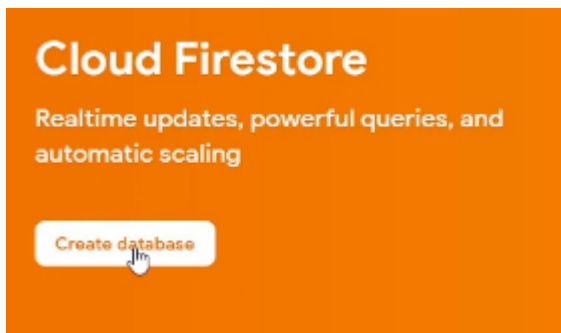
## 20. React 3 - Databáze firebase - založení databáze a její naplnění

Video: <https://youtu.be/iebHdwygXHE>

V projektu v levém menu klikneme v nabídce Build na Firestore Database



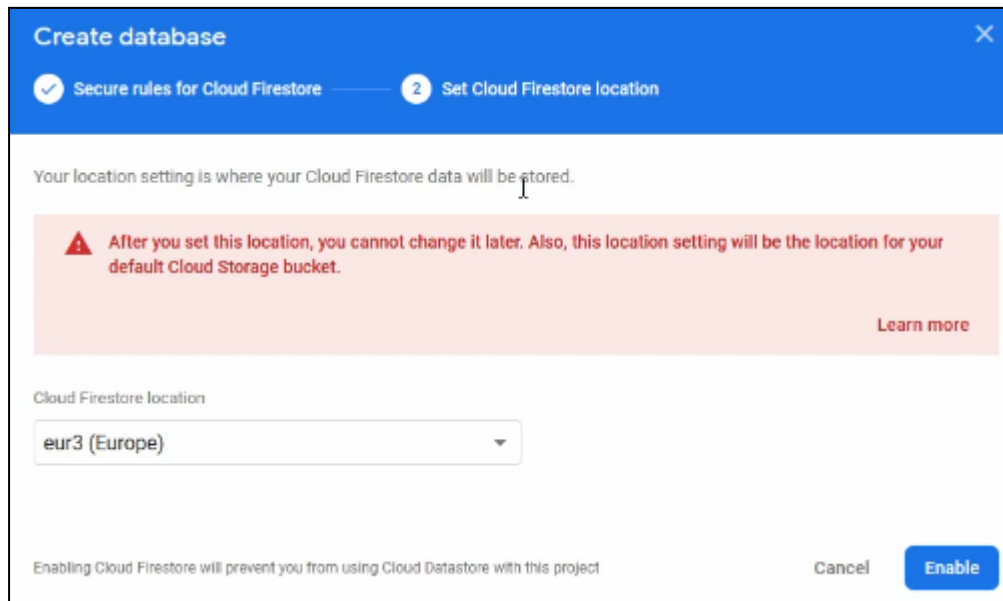
Dáme tlačítko Create database



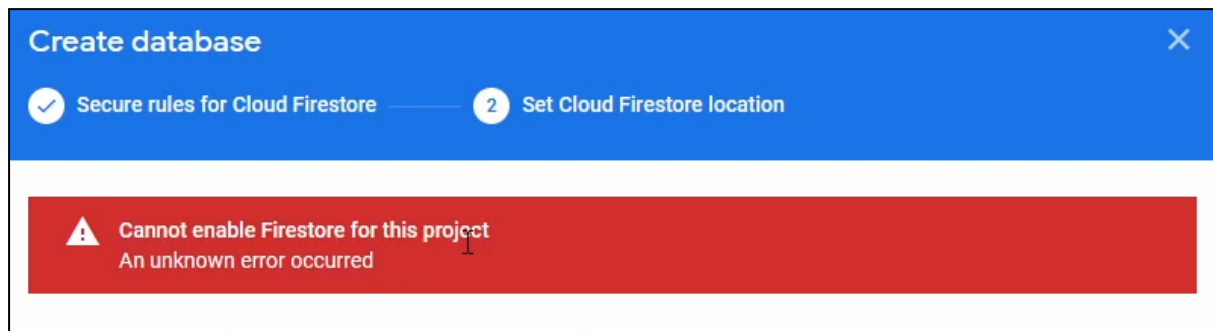
Dáme Start in production mode a klikneme na Next



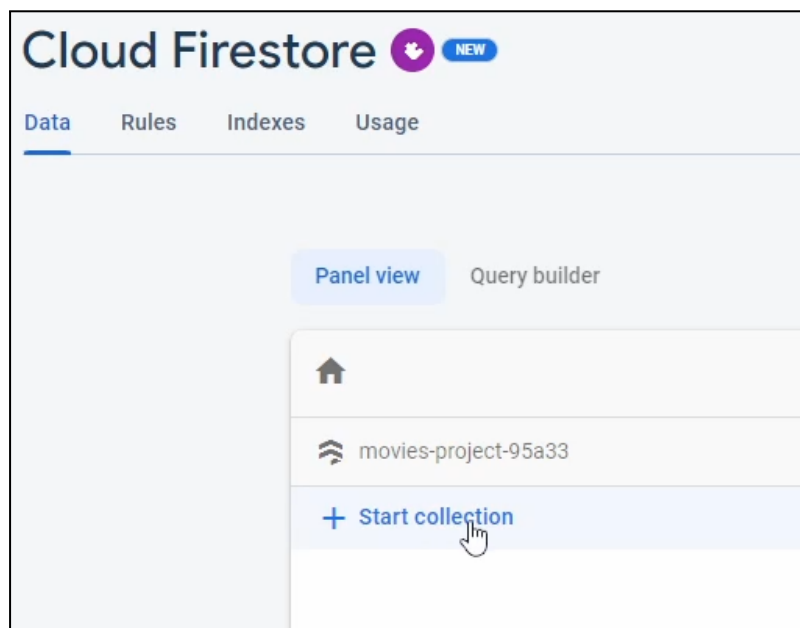
Vybereme location (můžete dát Europe) a dáte Enable



Pokud vyskočí tato hláška, tak je nejlepší změnit prohlížeč a vše provést v jiném prohlížeči.



Dostáváme se do databáze

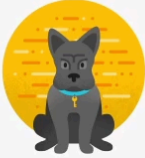


Nahoře dáme Rules a přepíšeme false na true

# Cloud Firestore

[Data](#)[Rules](#)[Indexes](#)[Usage](#)

[Edit rules](#)[Monitor rules](#)



Guard your data with rules that define who has access to it and how it is structured

[View the docs](#)

unpublished changes

[Publish](#)[Discard](#)

```
1 rules_version = '2';
2 service cloud.firestore {
3   match /databases/{database}/documents {
4     match /{document=**} {
5       allow read, write: if true;
6     }
7   }
8 }
```

Přidáme collection a nazveme ji movies

## Start a collection

1 Give the collection an ID

2 Add its first document

Parent path

/

Collection ID ?

Cancel

Next

Přidáme první dva filmy. Takto může vypadat první film.

Document ID ?

PwOjRlVyzwQPEkxihUnG

| Field  | Type   | Value    |
|--------|--------|----------|
| title  | string | 4gauneři |
| minage | number | 16       |
| time   | number | 121      |

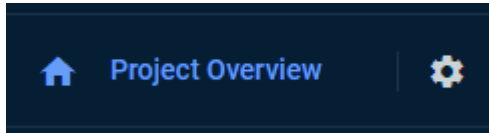
Takto vypadá výsledná databáze

|                              |                                                |                                                                                |
|------------------------------|------------------------------------------------|--------------------------------------------------------------------------------|
| 🏠 > movies > PZsy2EUb9St2... |                                                |                                                                                |
| 📁 movies-project-95a33       | 📁 movies                                       | 📄 PZsy2EUb9St2Mk8VQeEa                                                         |
| + Start collection           | + Add document                                 | + Start collection                                                             |
| movies >                     | PZsy2EUb9St2Mk8VQeEa ><br>PwOjRlVyzwQPEkxihUnG | + Add field<br>minage: 12<br>time: 135<br>title: "Harry Potter a kámen mudrců" |

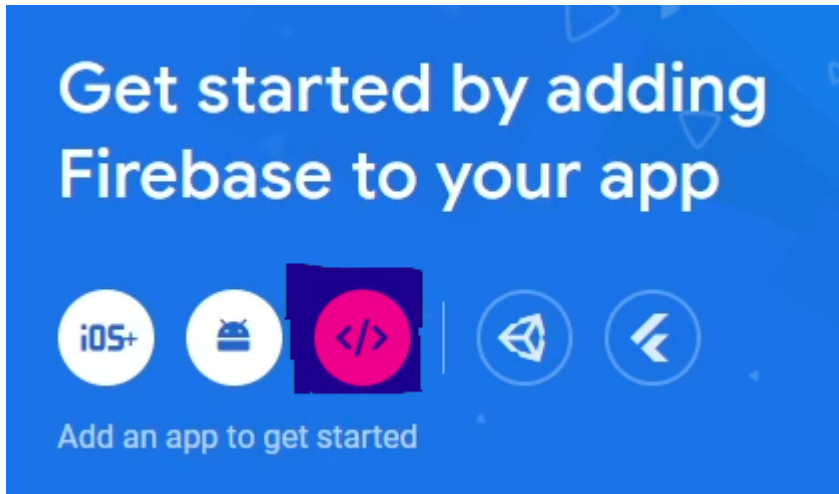
## 21. React 3 - Databáze firebase - vytvoření aplikace a nastavení config souboru

Video: <https://youtu.be/KDL1RMivKkg>

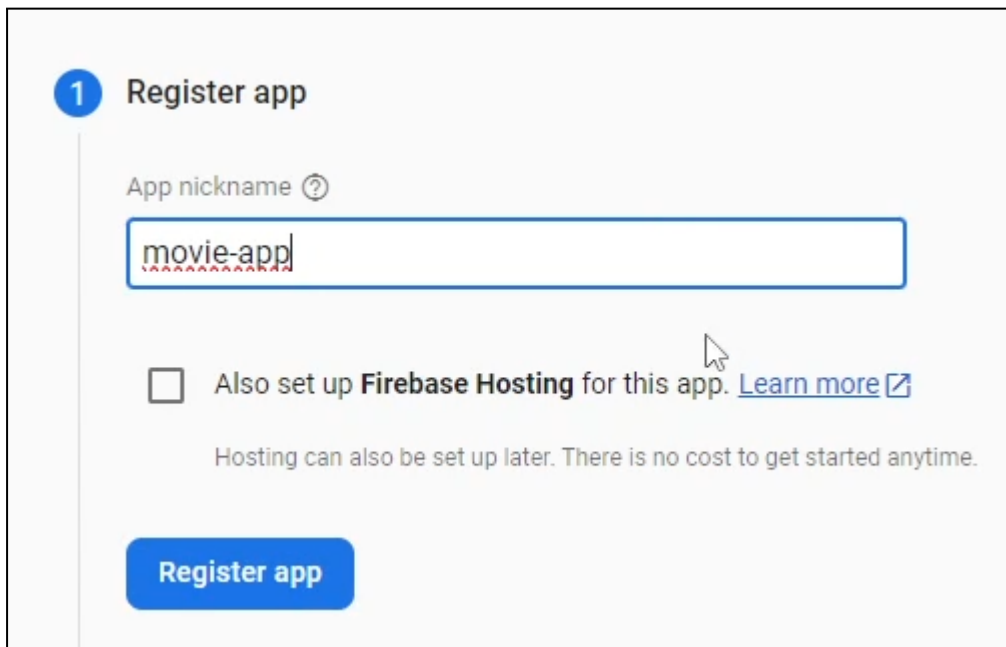
Klikneme na Project Overview



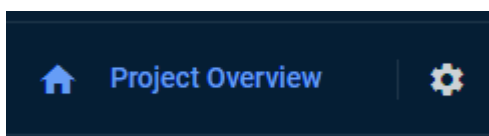
To nás dostane na úvodní stranu projektu, kde klikneme na ikonku webové aplikace



Zadáme libovolný název a dáme Register app.



Můžeme vše zavřít a klikneme na ikonu ozubeného kolečka vedle Project Overview.



Zde vidíme všechny potřebné údaje. Nás hlavně zajímá `const firebaseConfig`. Ten si zkopírujeme.

### SDK setup and configuration

☒ npm ☐ CDN ☐ Config

If you're already using [npm](#) and a module bundler such as [webpack](#) or [Rollup](#), you can run the following command to install the latest SDK ([Learn more](#)):

```
$ npm install firebase
```

Then, initialize Firebase and begin using the SDKs for the products you'd like to use.

```
// Import the functions you need from the SDKs you need
import { initializeApp } from "firebase/app";
// TODO: Add SDKs for Firebase products that you want to use
// https://firebase.google.com/docs/web/setup#available-libraries

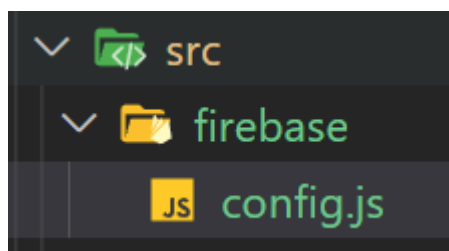
// Your web app's Firebase configuration
const firebaseConfig = {
  apiKey: "AIzaSyB1B0zcv137sMGa7UGk9T9qYosQgp2gR5s",
  authDomain: "movies-project-95a33.firebaseio.com",
  projectId: "movies-project-95a33",
  storageBucket: "movies-project-95a33.appspot.com",
  messagingSenderId: "1081775193844",
  appId: "1:1081775193844:web:35ce88970238344af2e8a1"
};

// Initialize Firebase
const app = initializeApp(firebaseConfig);
```

Nyní nainstalujeme v terminálu firebase pomocí příkazu:

**npm install firebase@8.5**

Ve složce `src` založíme složku `firebase` a vní soubor `config.js`



## config.js

Takto bude vypadat konfigurační soubor. Můžete ho používat stále dokola. Jen se bude měnit firebaseConfig s přihlašovacími údaji, když se budete hlásit do jiné aplikace.

```
import firebase from "firebase/app"
import "firebase/firestore"

const firebaseConfig = {
  apiKey: "AIzaSyB1BOzcv137sMGa7UGk9T9qYosQgp2gR5s",
  authDomain: "movies-project-95a33.firebaseio.com",
  projectId: "movies-project-95a33",
  storageBucket: "movies-project-95a33.appspot.com",
  messagingSenderId: "1081775193844",
  appId: "1:1081775193844:web:35ce88970238344af2e8a1"
}

// počáteční nastavení firebase (init)
firebase.initializeApp(firebaseConfig)

// počáteční nastavení služeb (services)
const projectFirestore = firebase.firestore()

export { projectFirestore }
```

## App.js

Do App.js projectFirestore naimportujeme.

```
import { projectFirestore } from "../firebase/config"

const App = () => {
  return (
    <div>App</div>
  )
}

export default App
```

## 22. React 3 - Databáze firebase - Řešíme chybu a instalujeme verzi 8.5

Video: <https://youtu.be/EWWDH6UbxIE>

V terminálu dáme tento příkaz, kterým nainstalujeme firebase verzi 8.5  
**npm install firebase@8.5**

V souboru package.json bychom měli vidět firebase verze 8.5

```
"dependencies": {  
  "@testing-library/jest-dom": "^  
  "@testing-library/react": "^13.  
  "@testing-library/user-event":  
  "firebase": "8.5",  
  "react": "^18.2.0",  
  "react-dom": "^18.2.0",  
  "react-scripts": "5.0.1",  
  "web-vitals": "^2.1.4"
```

## 23. React 3 - Databáze firebase - vypisujeme data z databáze do stránky

Video: <https://youtu.be/SsZVDpITBBk>

### App.js

V useEffectu provedeme načtení dat z databáze. Přihlásíme se do collection s názvem movies a získáme z něj všechny údaje a ty uložíme do proměnné snapshot. Pokud je snapshot prázdný (empty je true), tak se do proměnné error nastaví text "Žádné filmy k vypsání". Pokud snapshot není prázdný (obsahuje filmy), tak ze snapshotu vytáhneme postupně documenty (docs) a k datům přidáme vždy id a tento nový objekt (id + data z documentu) uložíme jako nový objekt do result. To uděláme se všemi filmy. Na konci už jen obsah result uložíme do proměnné data pomocí setData(result).

Druhá část je vypsání obsahu proměnné data pomocí funkce map.

```
import { projectFirestore } from "../firebase/config"
import { useState, useEffect } from "react"

const App = () => {
  const [data, setData] = useState([])
  const [error, setError] = useState(false)

  useEffect( () => {
    projectFirestore.collection("movies").get().then( (snapshot) => {

      if (snapshot.empty){
        setError("Žádné filmy k vypsání")
      } else {
        let result = []
        snapshot.docs.forEach( (oneMovie) => {
          result.push( {id: oneMovie.id, ...oneMovie.data()} )
        } )
        setData(result)
      }
    }).catch( (err) => {
      setError(err.message)
    })
  }, [])

  return <div>
    {error && <p>{error}</p>}
    {data.map( (oneMovie) => {
      const {id, title, minage, time} = oneMovie

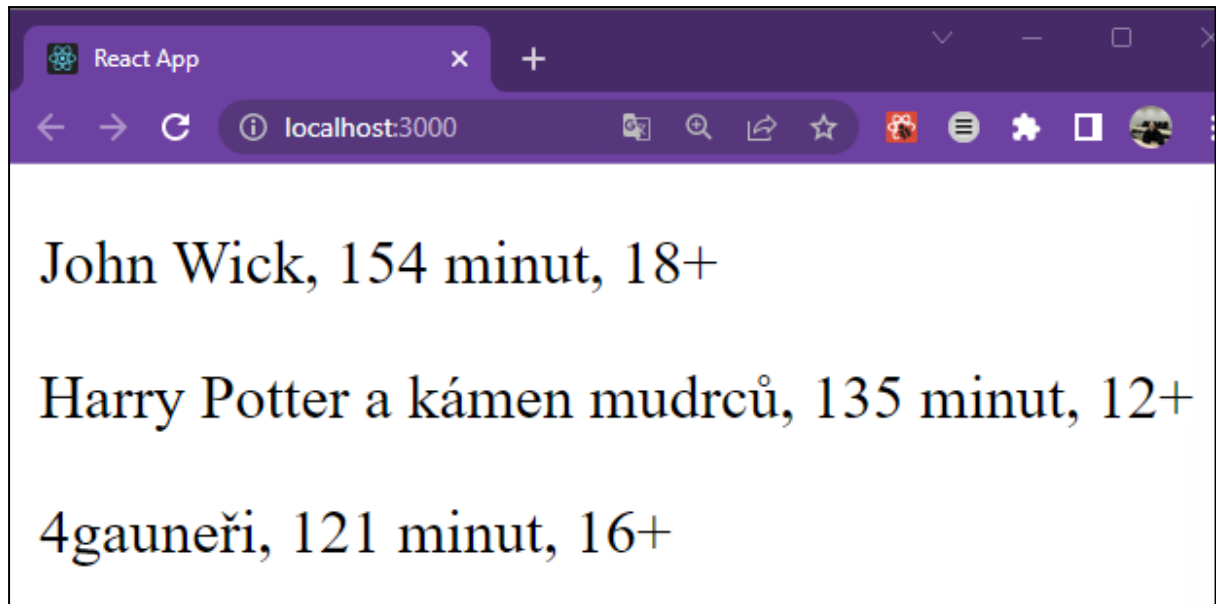
      return <div key={id}>
        <p>{title}, {time} minut, {minage}</p>
      }
    )}
```

```
    </div>

    } )}
  </div>
}

export default App
```

Výsledek na stránce



## 24. React 3 - Databáze firebase - stylujeme aplikaci

Video: <https://youtu.be/0CFE4gNC4DQ>

### App.js

Doplníme dvě classy: all-movies a one-movie

```
return <div className="all-movies">
  {error && <p>{error}</p>}
  {data.map( (oneMovie) => {
    const {id, title, minage, time} = oneMovie

    return <div key={id} className="one-movie">
      <p>{title}, {time} minut, {minage}</p>
    </div>

  } )}
</div>
```

### index.css

```
body {
  margin: 0;
  padding: 0;
  background-color: #141414;
  color: #e5e5e5;
}

.all-movies {
  width: 100%;
  height: 100vh;
  display: flex;
  flex-direction: column;
  justify-content: center;
  align-items: center;
}

.one-movie {
  font-size: 20px;
  border: 2px solid #e50914;
  width: 400px;
  padding: 5px 10px;
  margin-bottom: 10px;
  text-align: left;
}
```

Výsledek na stránce

John Wick, 154 minut, 18+

Harry Potter a kámen mudrců, 135 minut, 12+

4gauneři, 121 minut, 16+

## 25. React 3 - Databáze firebase - mažeme filmy z databáze

Video: <https://youtu.be/lsfr3lixXKs>

App.js

Do return, které vypisuje všechny filmy (uvnitř map) přidáme tlačítko. Když se na tlačítko klikne, tak se spustí funkce deleteMovie a pošle se do ní id filmu, na který jsme klikli.

```
return <div key={id} className="one-movie">
  <p>{title}, {time} minut, {minage}</p>
  <button type="button" onClick={ () => deleteMovie(id)
}>Smazat</button>
</div>
```

Někam nahoru přidáme funkci deleteMovie, ve které řekneme, že se chceme přihlásit do collection movies, zvolit document s daným id a ten vymazat.

```
const deleteMovie = (id) => {
  projectFirestore.collection("movies").doc(id).delete()
}

return <div className="all-movies">
  {error && <p>{error}</p>}
  {data.map( (oneMovie) => {
    const {id, title, minage, time} = oneMovie

    return <div key={id} className="one-movie">
      <p>{title}, {time} minut, {minage}</p>
      <button type="button" onClick={ () => deleteMovie(id)
    }>Smazat</button>
    </div>

    } )}
</div>
```

Pokud klikneme na tlačítko Smazat, tak se film z databáze smaže, ale na stránce zůstane, protože se stránka znovu nenačtetla. Jakmile stránku znovu načtete, tak film zmizí. To musíme dále vyřešit.

Harry Potter a kámen mudrců, 135 minut, 12+

Smazat

4gauneři, 121 minut, 16+

Smazat

## 26. React 3 - Databáze firebase - změny se dějí v reálném čase

Video: <https://youtu.be/0P-tyN4-ck>

Problém - když smažeme na stránce film, tak se sice vymaže z databáze, ale na stránce stále zůstává až do opětovného načtení stránky. Proč tomu tak je? Protože data z databáze se ukládají do tzv. local state - něco jako sklad v našem prohlížeči. A z něj se filmy načítají do stránky. Při mazání v databázi se ale local state neupdatuje, takže se stále načítají ty samé filmy.

Řešení - na collection movies navážeme posluchač událostí s názvem onSnapshot. Ten se spustí ve chvíli, kdy se v databázi cokoliv změní. A pošle zpět snímek databáze (současný stav databáze). Fialová část kódu ukazuje, jak se řeší zachytávání chyb u onSnapshot. Už zde není catch.

Ted' nám ale vzniká v pozadí kódu chyba. Posluchač událostí je navázaný na databázi. Pokud bychom přešli na jinou stránku (např. úvodní strana, kde můžeme přidávat nové filmy), tak i na úvodní straně bude docházet ke změně databáze. A to spustí posluchač události, který spustí kód, který máme zde níže (i když na té stránce vůbec nejsme). A u části setData(result) to vyhodí chybu, protože nic takového na stránce není (jsme na úvodní straně). Aby k tomu nedocházelo, tak se musí v pravý čas posluchače události zbavit. A to uděláme pomocí cleanup function. Náš kód nám vrátí funkci, díky které můžeme posluchač události dát pryč. Tuto funkci uložíme do proměnné unsubscribe. Úplně dole ji zavoláme a posluchač tím odstraníme.

```
import { projectFirestore } from "../firebase/config"
import { useState, useEffect } from "react"

const App = () => {
  const [data, setData] = useState([])
  const [error, setError] = useState(false)

  useEffect( () => {
    const unsubscribe =
projectFirestore.collection("movies").onSnapshot( (snapshot) => {

      if (snapshot.empty){
        setError("Žádné filmy k vypsání")
        setData([])
      } else {
        let result = []
        snapshot.docs.forEach( (oneMovie) => {
          result.push( {id: oneMovie.id, ...oneMovie.data()} )
        } )
        setData(result)
        setError("")
      }
    }, err => setError(err.message) )
  })
}
```

```
return () => unsubscribe()
```

```
}, [])
```

## 27. React 3 - Databáze firebase - stylujeme tlačítka

Video: <https://youtu.be/1-iXdePa8Vk>

index.css

Do index.css přidáme tyto styly.

```
.one-movie {
  font-size: 20px;
  border: 2px solid #e50914;
  width: 500px;
  padding: 5px 10px;
  margin-bottom: 10px;
  text-align: left;

  display: flex;
  flex-direction: row;
  justify-content: space-between;
  align-items: center;
}

.one-movie button {
  margin-right: 10px;
  font-size: 16px;
  background-color: #e50914;
  color: #e5e5e5;
  border: none;
  padding: 5px 10px;
  cursor: pointer;
}
```

## 28. React 3 - Databáze firebase - vytvoření formuláře pro přidávání údajů do databáze

Video: <https://youtu.be/F4xZ4v1esXM>

App.js

Pomocí useState vytvoříme tři proměnné, protože z formuláře budeme načítat tři údaje.

```
import { projectFirestore } from "../firebase/config"
import { useState, useEffect } from "react"

const App = () => {
  const [data, setData] = useState([])
  const [error, setError] = useState(false)
  // useState pro formulář
  const [movieTitle, setMovieTitle] = useState("")
  const [movieAge, setMovieAge] = useState(null)
  const [movieTime, setMovieTime] = useState(null)
```

**App.js**

Vytvoříme formulář a pomocí setMovieTitle, setMovieAge a setMovieTime budeme nastavovat do proměnných movieTitle, movieAge a movieTime hodnoty z formuláře. Jakmile se formulář odešle, tak se spustí funkce submitForm, kde vypneme defaultní chování formuláře (opětovné načtení stránky vypneme) a testově jsme si obsahy všech tří proměnných vypsali do konzole.

```
const deleteMovie = (id) => {
  projectFirestore.collection("movies").doc(id).delete()
}

const submitForm = (e) => {
  e.preventDefault()

  console.log(movieTitle)
  console.log(movieAge)
  console.log(movieTime)
}

return <div className="all-movies">
  <form onSubmit={submitForm}>
    <input
      type="text"
      onChange={ (e) => setMovieTitle( e.target.value ) }
      placeholder="Název filmu"
    /><br />
```

```

      <input
        type="number"
        onChange={ (e) => setMovieAge(e.target.value) }
        placeholder="Minimální věk"
        min="0"
      /><br />

      <input
        type="number"
        onChange={ (e) => setMovieTime(e.target.value) }
        placeholder="Doba trvání"
        min="0"
      /><br />

      <input type="submit" value="Přidat" />
    </form>

    {error && <p>{error}</p>}
    {data.map( (oneMovie) => {
      const {id, title, minage, time} = oneMovie

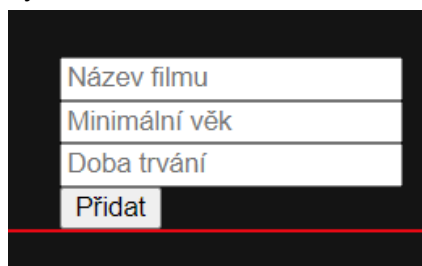
      return <div key={id} className="one-movie">
        <p>{title}, {time} minut, {minage}</p>
        <button type="button" onClick={ () => deleteMovie(id)
      }>Smazat</button>
      </div>

    } )}
  </div>
}

export default App

```

Výsledek na stránce



The screenshot shows a web application interface with a dark background. It features a form with three stacked input fields. The first field is labeled 'Název filmu', the second 'Minimální věk', and the third 'Doba trvání'. Below these fields is a button labeled 'Přidat'. A red horizontal line is visible at the bottom of the form area.

## 29. React 3 - Databáze firebase - zprovoznění formuláře pro přidání dat do databáze

Video: [https://youtu.be/Trw\\_0xiv\\_7s](https://youtu.be/Trw_0xiv_7s)

### App.js

Z dat, které získáme z formuláře vytvoříme nový objekt s názvem newMovie a ten pošleme do databáze.

Nastavíme všechny proměnné na prázdné stringy a pomocí value jejich hodnoty propíšeme do formuláře = po odeslání formuláře se políčka promažou.

```
const submitForm = async (e) => {
  e.preventDefault()

  const newMovie = {title: movieTitle, minage: movieAge, time:
movieTime}

  try {
    await projectFirestore.collection("movies").add(newMovie)
    setMovieTitle("")
    setMovieAge("")
    setMovieTime("")
  } catch (err) {
    setError("Film nebyl přidán " + err.message)
  }
}

return <div className="all-movies">
  <form onSubmit={submitForm}>
    <input
      type="text"
      onChange={ (e) => setMovieTitle( e.target.value ) }
      placeholder="Název filmu"
      value={movieTitle}
    /><br />

    <input
      type="number"
      onChange={ (e) => setMovieAge(e.target.value) }
      placeholder="Minimální věk"
      min="0"
      value={movieAge}
    /><br />

    <input
```

```
    type="number"
    onChange={ (e) => setMovieTime(e.target.value) }
    placeholder="Doba trvání"
    min="0"
    value={movieTime}
  /><br />

  <input type="submit" value="Přidat" />
</form>
```

## 30. React 3 - Databáze firebase - stylování formuláře

Video: <https://youtu.be/Sbpl0aNnzjo>

### App.js

Přidáme classy

```
return <div className="all-movies">
  <form onSubmit={handleSubmit} className="form">
    <input
      type="text"
      className="input"
      onChange={ (e) => setMovieTitle( e.target.value ) }
      placeholder="Název filmu"
      value={movieTitle}
    />

    <input
      type="number"
      className="input"
      onChange={ (e) => setMovieAge(e.target.value) }
      placeholder="Minimální věk"
      min="0"
      value={movieAge}
    />

    <input
      type="number"
      className="input"
      onChange={ (e) => setMovieTime(e.target.value) }
      placeholder="Doba trvání"
      min="0"
      value={movieTime}
    />

    <input type="submit" value="Přidat" />
  </form>
```

## index.css

Tyto styly přidáme do index.css

```
/* Formulář */
.form {
  margin-bottom: 20px;
  display: flex;
  flex-direction: column;
}

.input {
  font-size: 16px;
  margin-bottom: 10px;
  background-color: #141414;
  border: 2px solid #e5e5e5;
  padding: 3px 10px;
  color: #e5e5e5;
}

.input::placeholder {
  color: #848383;
}

.form input[type="submit"] {
  background-color: #e50914;
  color: #e5e5e5;
  border: none;
  font-size: 16px;
  padding: 5px 10px;
  cursor: pointer;
}
```

Název filmu

Minimální věk

Doba trvání

Přidat

Karlík a továrna na čokoládu, 158 minut, 12+

Smazat

John Wick, 121 minut, 16+

Smazat

Harry Potter a kámen mudrců, 151 minut, 14+

Smazat

## 31. React 3 - Databáze firebase - závěrečné video

Video: <https://youtu.be/uEkod5XRyWE>

Karlík a továrna na čokoládu, 158 minut, 12+

John Wick, 121 minut, 16+

Harry Potter a kámen mudrců, 151 minut, 14+

## **32. React 3 - Databáze firebase 2 - založení nového projektu**

Video: <https://youtu.be/ZYzFifjP58Q>

Dopředu víme, že budeme potřebovat react-router pro tvorbu vícestránkového webu. Proto ve složce, kde spouštíme npm start, tak dáme tento příkaz:

**npm install react-router-dom@6**

Také budeme potřebovat napojení na databázi firebase, takže firebase nainstalujeme. Použijeme tento příkaz:

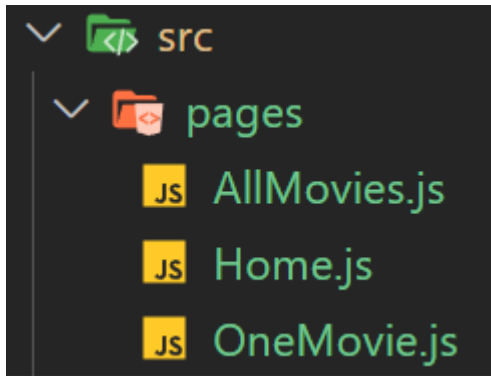
**npm install firebase@8.5**

Jinak projekt připravíme jako vždy. Ve složce src zůstane jen App.js, index.css a index.js.

### 33. React 3 - Databáze firebase 2 - tvoříme vícestránkový web

Video: <https://youtu.be/nDmJumVL8zM>

Ve složce src vytvoříme složku pages a do ní umístíme tři soubory (tři stránky).



Všechny obsahují jen obyčejnou komponentu.

#### Home.js

```
const Home = () => {  
  return (  
    <div>Home</div>  
  )  
}  
  
export default Home
```

#### AllMovies.js

```
const AllMovies = () => {  
  return (  
    <div>AllMovies</div>  
  )  
}  
  
export default AllMovies
```

#### OneMovie.js

```
const OneMovie = () => {  
  return (  
    <div>OneMovie</div>  
  )  
}  
  
export default OneMovie
```

## App.js

Naimportujeme vše, co potřebujeme k vytvoření vícestránkového webu a také naše tři stránky.

V return vytvoříme vícestránkový web.

```
import { BrowserRouter, Routes, Route } from "react-router-dom"
import Home from "../pages/Home"
import AllMovies from "../pages/AllMovies"
import OneMovie from "../pages/OneMovie"

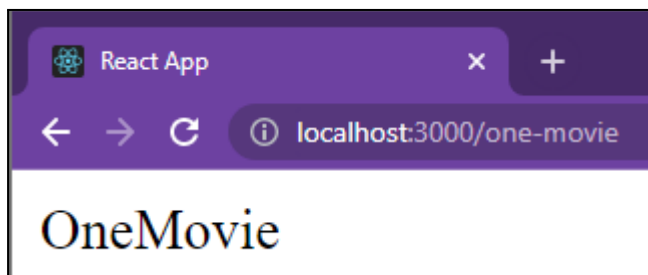
const App = () => {
  return <BrowserRouter>
    <Routes>

      <Route path="/" element={ <Home /> } />
      <Route path="all-movies" element={ <AllMovies /> } />
      <Route path="one-movie" element={ <OneMovie /> } />

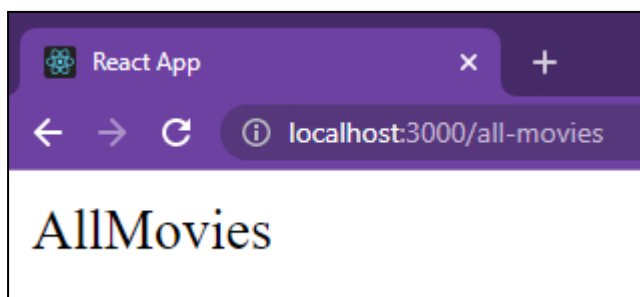
    </Routes>
  </BrowserRouter>
}

export default App
```

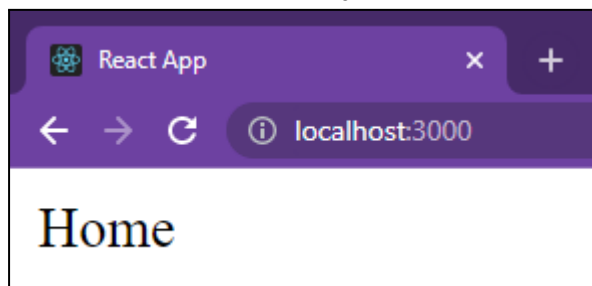
Pokud někdo zadá /one-movie



Pokud někdo zadá /all-movies



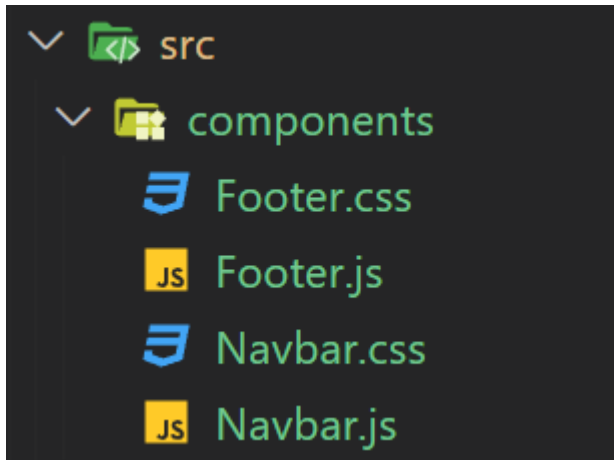
Pokud někdo zadá / nebo jen adresu úvodní strany (bez /)



## 34. React 3 - Databáze firebase 2 - tvoříme hlavičku a patičku

Video: [https://youtu.be/lmOK9\\_ePTWw](https://youtu.be/lmOK9_ePTWw)

Ve složce src založíme složku components a naplníme ji soubory pro navigaci a patičku.



Navbar.js

```
import "../Navbar.css"
import { NavLink } from "react-router-dom"

const Navbar = () => {
  return <header>
    <nav>
      <NavLink to="/">Domů</NavLink>
      <NavLink to="all-movies">Filmy</NavLink>
    </nav>
  </header>
}

export default Navbar
```

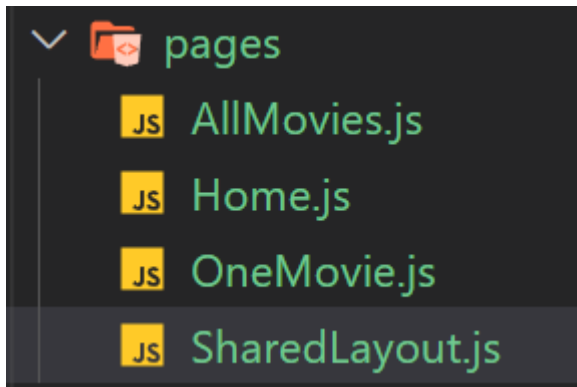
Footer.js

```
import "../Footer.css"

const Footer = () => {
  return <footer>
    <p>Patička</p>
  </footer>
}

export default Footer
```

Ve složce pages vytvoříme soubor SharedLayout.js



SharedLayout.js

```
import Navbar from "../components/Navbar"
import Footer from "../components/Footer"

const SharedLayout = () => {
  return <>
    <Navbar />

    <Footer />
  </>
}

export default SharedLayout
```

V prohlížeči zatím žádnou změnu nevidíme!

## 35. React 3 - Databáze firebase 2 - dokončujeme vícestránkový web

Video: [https://youtu.be/s\\_MJEZEmAic](https://youtu.be/s_MJEZEmAic)

### SharedLayout.js

Naimportujeme Outlet a dáme ho mezi komponenty Navbar a Footer. Outlet je obsah daného elementu z App.js (viz níže).

```
import Navbar from "../components/Navbar"
import Footer from "../components/Footer"
import { Outlet } from "react-router-dom"

const SharedLayout = () => {
  return <>
    <Navbar />
    <Outlet />
    <Footer />
  </>
}

export default SharedLayout
```

### App.js

Naimportujeme SharedLayout a řekneme, že pokud někdo zadá např. /all-movies, tak se zobrazí obsah SharedLayout (obalujícího elementu) a místo Outlet se do SharedLayout dá obsah komponenty AllMovies. A takto to funguje u všech stránek. Jen úvodní strana je specifická. Slovo index znamená path obalujícího elementu (takže path="/")

```
import OneMovie from "../pages/OneMovie"
import SharedLayout from "../pages/SharedLayout"

const App = () => {
  return <BrowserRouter>
    <Routes>

      <Route path="/" element={ <SharedLayout /> } >
        <Route index element={ <Home /> } />
        <Route path="all-movies" element={ <AllMovies /> } />
        <Route path="one-movie" element={ <OneMovie /> } />
      </Route>

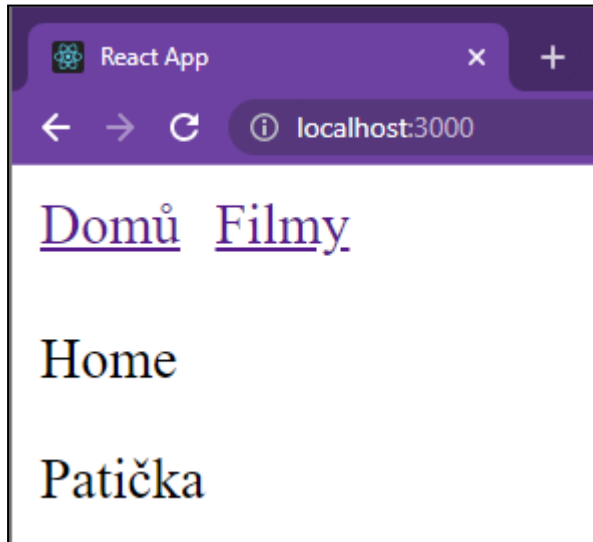
    </Routes>
  </BrowserRouter>
}
```

## Navbar.css

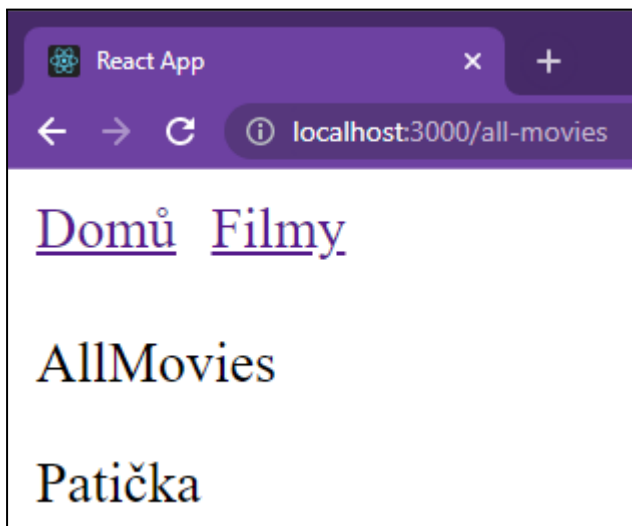
Jedná se jen o kosmetické úpravy

```
nav a {  
  margin-right: 10px;  
  display: inline-block;  
  margin-bottom: 20px;  
}
```

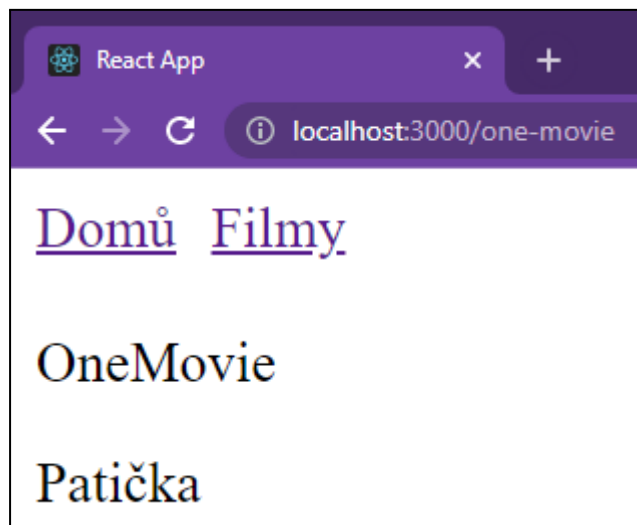
V prohlížeči se zadá úvodní strana



V prohlížeči se zadá /all-movies



V prohlížeči se zadá /one-movie



## 36. React 3 - Databáze firebase 2 - parametry v url adrese stránky

Video: [https://youtu.be/U7VuP\\_p6zU0](https://youtu.be/U7VuP_p6zU0)

### App.js

Do url adresy u one-movie doplníme ještě parametr. Zde jsme ho pojmenovali movieId.

```
return <BrowserRouter>
  <Routes>

    <Route path="/" element={ <SharedLayout /> } >
      <Route index element={<Home />} />
      <Route path="all-movies" element={<AllMovies />} />
      <Route path="one-movie/:movieId" element={<OneMovie />} />
    </Route>

  </Routes>
</BrowserRouter>
```

### OneMovie.js

Pokud se načte komponenta OneMovie, tak chceme pomocí useParams vytáhnout parametry z url adresy. Konkrétní hodnotu parametru uložíme do proměnné movieId a tu vypíšeme do stránky.

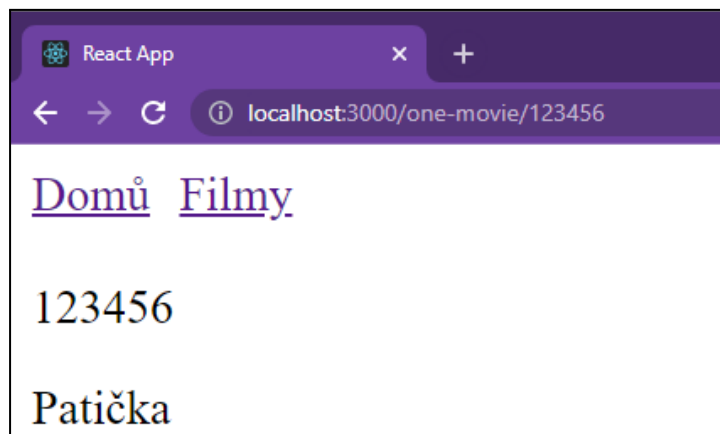
```
import { useParams } from "react-router-dom"

const OneMovie = () => {
  const { movieId } = useParams()

  return (
    <div>{ movieId }</div>
  )
}

export default OneMovie
```

Pokud někdo zadá adresu /one-movie/123456, tak se zobrazí stránka takto



## 37. React 3 - Databáze firebase 2 - založení databáze

Video: <https://youtu.be/ABe34LYyigg>

1. vytvoříme projekt,
2. zadáme libovolný název projektu,
3. vypneme Google Analytics,
4. projekt se vytvoří,
5. v levém menu dáme Build -> Firestore Database
6. klikneme na Create database
7. dáme Start in production mode
8. vybereme zemi; pokud se nyní zobrazí chybová hláška, tak se přesuneme do jiného prohlížeče,
9. nahoře zvolíme záložky Rules a přepíšeme false na true



The screenshot shows the Firebase Rules editor interface. At the top, there are three tabs: 'Rules' (selected), 'Indexes', and 'Usage'. Below the tabs, there are two buttons: 'Edit rules' (highlighted in blue) and 'Monitor rules'. The main area displays a list of rules on the left and the rule configuration on the right. The list on the left shows two entries: one with a star icon and timestamp 'Today • 9:42 PM', and another with a circle icon and timestamp 'Today • 9:41 PM'. The rule configuration on the right shows the following code:

```
1 rules_version = '2';
2 service cloud.firestore {
3   match /databases/{database}/documents {
4     match /{document=**} {
5       allow read, write: if true;
6     }
7   }
8 }
```

10. naplníme databázi daty o filmech

|                    |                       |                    |
|--------------------|-----------------------|--------------------|
| + Start collection | + Add document        | + Start collection |
| movies >           | 4ZRSwm2h5YQBAGLZzd6 > | + Add field        |
|                    | ttXEQVGuB1xMS9leueli  | minage: 16         |
|                    |                       | time: 121          |
|                    |                       | title: "John Wick" |

11. založíme aplikaci na úvodní straně projektu pomocí této ikonky

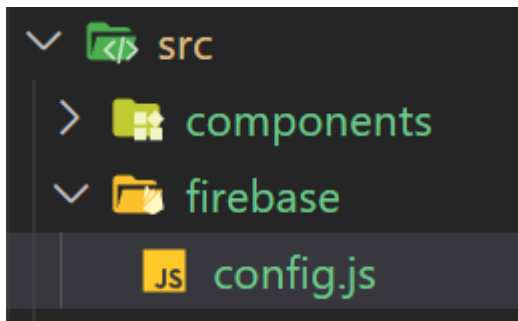


12. musíme se propracovat k ověřovacím údajům

```
const firebaseConfig = {  
  apiKey: "AIzaSyCoW9aTKw0Tusj5F0x4DM_xU8LBV1JXuks",  
  authDomain: "movies-project-2.firebaseio.com",  
  projectId: "movies-project-2",  
  storageBucket: "movies-project-2.appspot.com",  
  messagingSenderId: "71229491632",  
  appId: "1:71229491632:web:d087a29ba04c7c5d636e75"  
};
```



Ve složce src vytvoříme složku firebase a v ní soubor config.js



### config.js

```
import firebase from "firebase/app"
import "firebase/firestore"

const firebaseConfig = {
  apiKey: "AIzaSyCoW9aTKwOTusj5F0x4DM_xU8LBV1JXuks",
  authDomain: "movies-project-2.firebaseio.com",
  projectId: "movies-project-2",
  storageBucket: "movies-project-2.appspot.com",
  messagingSenderId: "71229491632",
  appId: "1:71229491632:web:d087a29ba04c7c5d636e75"
};

// počáteční inicializace
firebase.initializeApp(firebaseConfig)

// počáteční initalizace služeb
const projectFirestore = firebase.firestore()

export { projectFirestore }
```

## 38. React 3 - Databáze firebase 2 - načítáme data z databáze

Video: <https://youtu.be/tqQRPQG9OI8>

### AllMovies.js

Nastavíme si dvě proměnné data a error pomocí useState. Error bude ukádat errorové hlášky a vypisovat v return do odstavce. Do data načteme informace z databáze. Postup je stejný, jako v předchozím projektu.

```
import { projectFirestore } from "../firebase/config"
import { useState, useEffect } from "react"
import { Link } from "react-router-dom"

const AllMovies = () => {
  const [data, setData] = useState([])
  const [error, setError] = useState("")

  useEffect( () => {
    projectFirestore.collection("movies").get().then( (snapshot) => {
      // console.log(snapshot.docs)

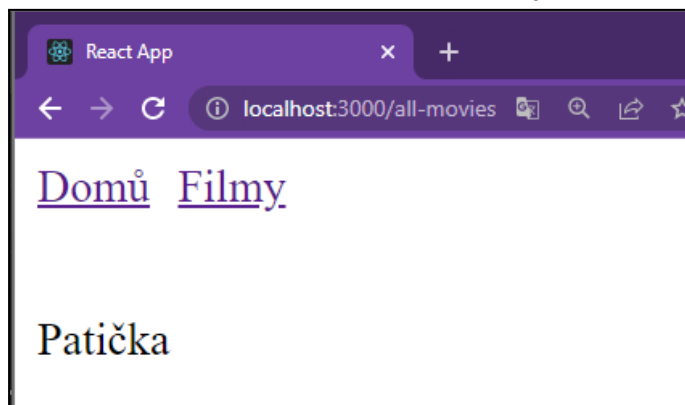
      if (snapshot.empty) {
        setError("Žádné filmy k vypsání")
      } else {
        let result = []
        snapshot.docs.forEach( (oneMovie) => {
          // console.log(oneMovie.data())
          result.push( {id: oneMovie.id, ...oneMovie.data()} )
        } )
        setData(result)
      }
    }).catch( (err) => {
      setError(err.message)
    } )

  }, [])

  return <section>
    {error && <p>{error}</p>}
  </section>
}

export default AllMovies
```

Na stránce **/all-movies** se nic nezobrazuje



## 39. React 3 - Databáze firebase 2 - vypisujeme informace z databáze do stránky

Video: <https://youtu.be/yaQFMHwx578>

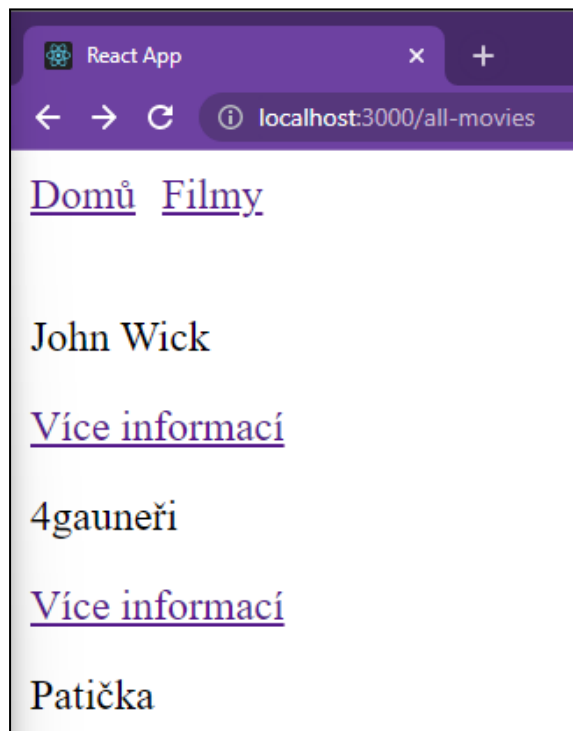
### App.js

Pomocí mapu vypíšeme z dat názvy filmů (title). Nejdříve jsme si z oneMovie (objekt, který reprezentuje jeden film) vytáhli pomocí destructuringu id, title, minage a time. Proměnné minage a time zde nebudeme potřebovat, tak je můžete smazat. Do stránky necháme vypsat div s identifikátorem id a dovnitř title filmu a odkaz s textem “Více informací”. Tento odkaz bude směřovat vždy na stránku /one-movie/idfilmu a idfilmu vezmeme z databáze (id filmu je uložené v proměnné id)

```
return <section>
  {error && <p>{error}</p>}
  {data.map( (oneMovie) => {
    const {id, title, minage, time} = oneMovie

    return <div key={id} >
      <p>{title}</p>
      <Link to={`/one-movie/${id}`}>Více informací</Link>
    </div>
  } )}
</section>
}
```

Jak vypadá stránka /all-movies



## 40. React 3 - Databáze firebase 2 - každý film má svou stránku

Video: <https://youtu.be/5eKhOCBeLeA>

### OneMovie.js

Zde postupujeme velice podobně jako v souboru AllMovies.js. Jen je zde rozdíl v tom, že vytahujeme z databáze konkrétní dokument s konkrétním ID (viz zelený kód).

```
import { useParams } from "react-router-dom"
import { projectFirestore } from "../firebase/config"
import { useState, useEffect } from "react"

const OneMovie = () => {
  const [data, setData] = useState({})
  const [error, setError] = useState(false)

  const { movieId } = useParams()

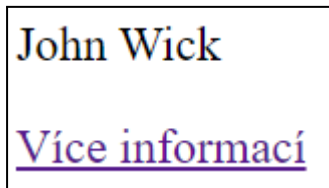
  useEffect( () => {
    projectFirestore.collection("movies").doc(movieId).get().then(
      (document) => {
        console.log(document)

        if (document.exists) {
          setData(document.data())
        } else {
          setError("Nenašli jsme tento film")
        }
      })
    .catch( (err) => {
      setError(err.message)
    })
  }, [])

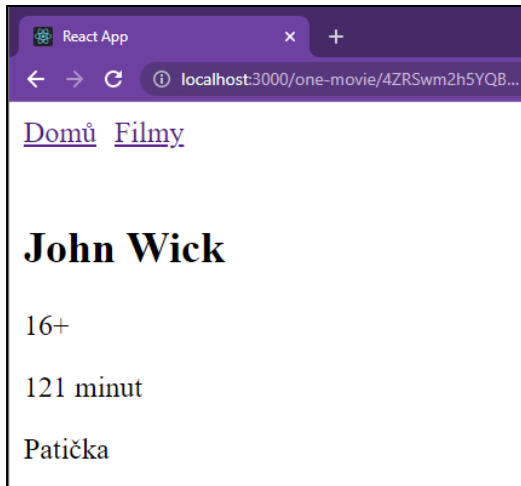
  return <section>
    <h1>{data.title}</h1>
    <p>{data.minage}</p>
    <p>{data.time} minut</p>
  </section>
}

export default OneMovie
```

Pokud někdo na stránce /all-movies klikne na více informací u Johna Wicka,



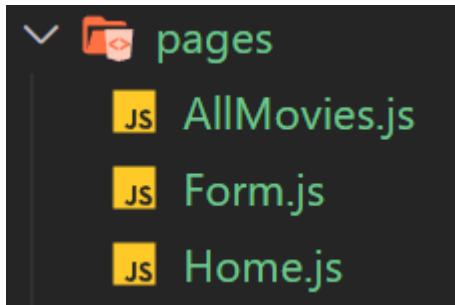
tak se dostáváme na stránku /one-movie/idfilmu, kde se nám vypíše informace o filmu.



## 41. React 3 - Databáze firebase 2 - příprava na formulář

Video: <https://youtu.be/HsPvp8A2lqE>

Ve složce pages vytvoříme soubor Form.js



### Form.js

Zatím ho naplníme jen obyčejnou komponentou

```
const Form = () => {  
  return (  
    <div>Form</div>  
  )  
}  
  
export default Form
```

### App.js

Naimportujeme komponentu Form

```
import Form from "../pages/Form"
```

Vytvoříme novou stránku, ve které se komponenta Form bude zobrazovat.

```
const App = () => {  
  return <BrowserRouter>  
    <Routes>  
  
      <Route path="/" element={ <SharedLayout /> } >  
        <Route index element={ <Home /> } />  
        <Route path="all-movies" element={ <AllMovies /> } />  
        <Route path="form" element={ <Form /> } />  
        <Route path="one-movie/:movieId" element={ <OneMovie /> } />  
      </Route>  
  
    </Routes>  
  </BrowserRouter>  
}
```

## Navbar.js

Přidáme si odkaz na novou stránku **/form** také do navigace na stránce.

```
const Navbar = () => {  
  return <header>  
    <nav>  
      <NavLink to="/">Domů</NavLink>  
      <NavLink to="all-movies">Filmy</NavLink>  
      <NavLink to="form">Přidání filmů</NavLink>  
    </nav>  
  </header>  
}
```

V navigaci vidíme třetí položku Přidání filmů, která slouží jako odkaz na stránku **/form**

[Domů](#) [Filmy](#) [Přidání filmů](#)

Form

Patička

## 42. React 3 - Databáze firebase 2 - tvorba formuláře

Video: <https://youtu.be/IMi5lpPo590>

### Form.js

Zde tvoříme formulář běžným způsobem. Vytvoříme formulář se 4 inputy (3 políčka a jedno odesílací tlačítko). Zachytáváme obsah políček pomocí události onChange a ukládáme je do proměnných movieTitle, movieAge a movieTime.

Po odeslání formuláře se spustí funkce submitForm a zatím zde jen testově vypisujeme obsah všech 3 proměnných.

```
import { useState } from "react"
import { projectFirestore } from "../firebase/config"

const Form = () => {
  const [movieTitle, setMovieTitle] = useState("")
  const [movieAge, setMovieAge] = useState(null)
  const [movieTime, setMovieTime] = useState(null)

  const submitForm = (event) => {
    event.preventDefault()

    console.log(movieTitle, movieAge, movieTime)

  }

  return <section>
    <form onSubmit={submitForm} >
      <input
        type="text"
        placeholder="Název filmu"
        onChange={ (e) => setMovieTitle(e.target.value) }
      />

      <input
        type="number"
        placeholder="Minimální věk"
        min="1"
        onChange={ (e) => setMovieAge(e.target.value) }
      />

      <input
        type="number"
        placeholder="Čas filmu"
        min="1"
        onChange={ (e) => setMovieTime(e.target.value) }
      />
    </form>
  </section>
}
```

```

    />

    <input type="submit" value="Přidat film" />

  </form>
</section>
}

export default Form

```

Takto vypadá stránka **/form**

React App

localhost:3000/form

[Domů](#) [Filmy](#) [Přidání filmů](#)

|             |               |
|-------------|---------------|
| Název filmu | Minimální věk |
| Čas filmu   | Přidat film   |

Patička

## 43. React 3 - Databáze firebase 2 - posíláme data do databáze a upravujeme předešlý kód

Video: <https://youtu.be/qGSldTdeQXE>

### Form.js

Z proměnných movieTitle, movieAge a movieTime vytvoříme nový objekt, který následně pošleme do databáze. Všechny tři proměnné nastavíme na prázdné hodnoty, aby se políčka na stránce po odeslání formuláře promazala. Hodnoty tří proměnných promítne do políček inputů pomocí value u každého inputu.

```
import { useState } from "react"
import { projectFirestore } from "../firebase/config"

const Form = () => {
  const [movieTitle, setMovieTitle] = useState("")
  const [movieAge, setMovieAge] = useState(null)
  const [movieTime, setMovieTime] = useState(null)

  const submitForm = async (event) => {
    event.preventDefault()

    console.log(movieTitle, movieAge, movieTime)

    const newMovie = {
      title: movieTitle,
      minage: parseInt(movieAge),
      time: parseInt(movieTime)
    }

    try {
      await projectFirestore.collection("movies").add(newMovie)
      setMovieTitle("")
      setMovieAge("")
      setMovieTime("")
    } catch (err) {
      console.log(err.message)
    }
  }

  return <section>
    <form onSubmit={submitForm}>
      <input
        type="text"
        placeholder="Název filmu"
        value={movieTitle} />
      <input
        type="text"
        placeholder="Věk"
        value={movieAge} />
      <input
        type="text"
        placeholder="Délka filmu"
        value={movieTime} />
      <button type="submit">Odeslat</button>
    </form>
  </section>
```

```

        onChange={ (e) => setMovieTitle(e.target.value) }
        value={movieTitle}
      />

      <input
        type="number"
        placeholder="Minimální věk"
        min="1"
        onChange={ (e) => setMovieAge(e.target.value)}
        value={movieAge}
      />

      <input
        type="number"
        placeholder="Čas filmu"
        min="1"
        onChange={ (e) => setMovieTime(e.target.value) }
        value={movieTime}
      />

      <input type="submit" value="Přidat film" />

    </form>
  </section>
}

export default Form

```

## Opravujeme předešlý kód

### OneMovie.js - dopíšeme movieId

```

useEffect( () => {
  projectFirestore.collection("movies").doc(movieId).get().then(
    (document) => {
      console.log(document)

      if (document.exists) {
        setData(document.data())
      } else {
        setError("Nenašli jsme tento film")
      }
    })
  ).catch( (err) => {
    setError(err.message)
  })
})

```

```
}, [movieId])
```

**OneMovie.js** - nepoužili jsme proměnnou `error`, tak ji vypíšeme do stránky, pokud chyba nastane

```
return <section>
  {error && <p>{error}</p>}
  <h1>{data.title}</h1>
  <p>{data.minage}+</p>
  <p>{data.time} minut</p>
</section>
```

**AllMovies.js** - v závorce jsme smazali `minage` a `time`, protože jsme je nepoužili

```
return <section>
  {error && <p>{error}</p>}
  {data.map( (oneMovie) => {
    const {id, title} = oneMovie

    return <div key={id} >
      <p>{title}</p>
      <Link to={`/one-movie/${id}`}>Více informací</Link>
    </div>
  } )}
</section>
```

## 44. React 3 - Databáze firebase 2 - mazání filmů

Video: <https://youtu.be/suEVMjEZtr4>

### AllMovies.js

Ke každému filmu přidáme tlačítko Smazat. Po kliknutí na tlačítko se spustí funkce deleteMovie, do které pošleme ID filmu, u kterého bylo na Smazat kliknuto. Tato funkce vymaže z databáze daný film.

```
const deleteMovie = (id) => {
  projectFirestore.collection("movies").doc(id).delete()
}

return <section>
  {error && <p>{error}</p>}
  {data.map( (oneMovie) => {
    const {id, title} = oneMovie

    return <div key={id} >
      <p>{title}</p>
      <Link to={`/one-movie/${id}`}>Více informací</Link>
      <button type="button" onClick={ () => deleteMovie(id)}>Smazat</button>
    </div>
  } )}
</section>
```

### AllMovies.js

Ale nebude nám to fungovat v reálném čase a film zmizí až po refreshnutí stránky. To můžeme vyřešit těmito úpravami, které přidají na databázi posluchač událostí, který se spustí, jakmile se v databázi něco změní.

```
useEffect( () => {
  const unsubscribe =
projectFirestore.collection("movies").onSnapshot( (snapshot) => {
  // console.log(snapshot.docs)

  if (snapshot.empty) {
    setError("Žádné filmy k vypsání")
  } else {
    let result = []
    snapshot.docs.forEach( (oneMovie) => {
      // console.log(oneMovie.data())
      result.push( {id: oneMovie.id, ...oneMovie.data()} )
    } )
    setData(result)
  }
})
}
```

```
    }, (err) => setError(err.message))

    return () => unsubscribe()
  }, [])
```

## 45. React 3 - Databáze firebase 2 - stylujeme aplikaci

Video: <https://youtu.be/INZOaHe270Q>

### Index.css

```
body {  
  margin: 0;  
  padding: 0;  
  background-color: #141414;  
  color: #e5e5e5;  
}
```

### Navbar.css

```
header {  
  display: flex;  
  flex-direction: row;  
  justify-content: center;  
}  
  
nav {  
  margin-top: 20px;  
  margin-bottom: 20px;  
}  
  
nav a {  
  margin-right: 10px;  
  display: inline-block;  
  text-decoration: none;  
  color: #e5e5e5;  
}  
  
/* .active {  
  color: #e50914;  
} */
```

### Footer.css

```
footer {  
  text-align: center;  
}
```

### Home.css

```
.home {  
  height: 50vh;  
  text-align: center;  
}
```

## AllMovies.css

```
.one-movie {
  display: flex;
  flex-direction: row;
  justify-content: space-between;
  align-items: center;
  border: 2px solid #e50914;
  padding-left: 10px;
  padding-right: 10px;
  margin: 10px 10px;
}

.one-movie a {
  text-decoration: none;
  color: #b92930;
}

.one-movie button {
  font-size: 14px;
  background-color: #e50914;
  color: #e5e5e5;
  border: none;
  padding: 5px 10px;
  cursor: pointer;
}
```

## Form.css

```
.form-section h1 {
  text-align: center;
}

.form-section form {
  display: flex;
  flex-direction: column;
  align-items: center;
}

.input {
  border: 2px solid #e50914;
  background-color: #141414;
  margin-bottom: 5px;
}
```

```
    outline: none;
    color: #e5e5e5;
}

.input::placeholder {
    color: #868686;
}

.form-section input[type="submit"] {
    background-color: #e50914;
    color: #e5e5e5;
    padding: 5px 10px;
    border: none;
    cursor: pointer;
    margin-top: 15px;
}
```

#### OneMovie.css

```
.one-movie-section {
    text-align: center;
    margin-bottom: 50px;
}
```

## 46. React 3 - Databáze firebase 2 - ještě dvě malé úpravy

Video: <https://youtu.be/q4PAhC8zhdk>

### AllMovies.css

```
.one-movie p {  
  width: 200px;  
}
```

Stránka /all-movies



### OneMovie.js

Naimportujeme Link pro odkaz

```
import { useParams, Link } from "react-router-dom"
```

### OneMovie.js

Vypíšeme dole pod informace odkaz, který nás vrátí zpět na stránku se všemi filmy.

```
return <section className="one-movie-section">  
  {error && <p>{error}</p>}  
  <h1>{data.title}</h1>  
  <p>{data.minage}+</p>  
  <p>{data.time} minut</p>  
  <Link exact to="/all-movies">Zpět na seznam filmů</Link>  
</section>
```

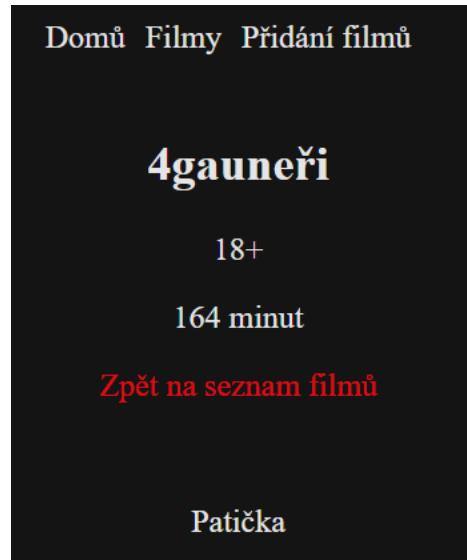
### OneMovie.css

Odkaz nastylujeme

```
.one-movie-section a {
```

```
text-decoration: none;  
color: #e50914;  
}
```

Takto stránka `/one-movie/idfilmu` vypadá



## 47. React 3 - Databáze firebase 2 - Závěrečné video

Video: <https://youtu.be/h5aJq0Owlxg>

### AllMovies.css

Malá úprava ve stylech

```
.one-movie {  
  display: flex;  
  flex-direction: row;  
  justify-content: space-between;  
  align-items: center;  
  border: 2px solid #e50914;  
  padding-left: 10px;  
  padding-right: 10px;  
  margin: 10px 10px;  
  
  width: 500px;  
  margin: 0 auto;  
}
```

Aby se na velkých monitorech boxíky nezobrazovaly přes celou šířku.

| Domů Filmy Přidání filmů     |                |        |
|------------------------------|----------------|--------|
| Harry Potter a kámen mudrců  | Více informací | Smazat |
| Karlík a továrna na čokoládu | Více informací | Smazat |
| Harry Potter a kámen mudrců  | Více informací | Smazat |
| 4gauneři                     | Více informací | Smazat |
| Patička                      |                |        |

## **48. React 3 - Databáze firebase 2 - závěrečné video kurzu**

Video: [https://youtu.be/5q\\_5ubQ64Ds](https://youtu.be/5q_5ubQ64Ds)

Díky za sledování kurzu